Carnegie Mellon University

95-702 Raft and Blockchain Consensus

A Definition from the Coulouris text

- A distributed system is one in which components located at networked computers communicate and coordinate their actions only by passing messages.

- Especially significant characteristics: concurrency of components, lack of global clock, independent failures of components

- Here, we will explore two example systems: Raft and Bitcoin

Two example distributed systems

Raft ← Used in lot of Data Centers (Replicate Data) (similar to Paxos)

Bitcoin ← Distributed System (Decentralized)

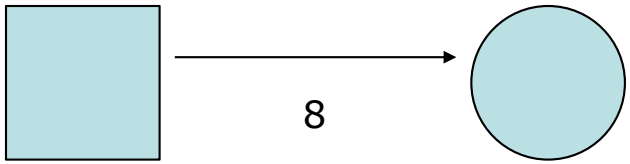
In both cases, the goal will be for the distributed components to reach consensus – but for different reasons.

Distributed Consensus: Let's start with a simple example

- Goal: Have a service store the number 8.

Fire and Forget Message Passing

(No Acknowledgement)

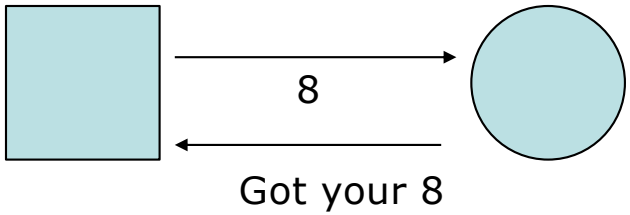


→ Maybe network is slow

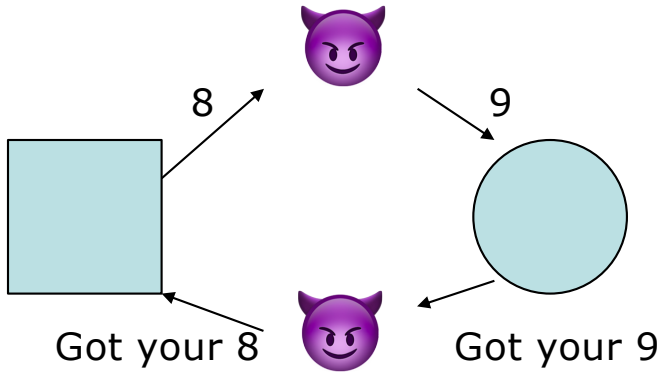
→ Maybe its routed elsewhere

→ Maybe server crashed

Request/Acknowledge provides some assurance



Security threat - the man in the middle attack



It is important to note that some protocols are designed to handle this use case and others are not. Raft, by design, makes no attempt to address this use case.

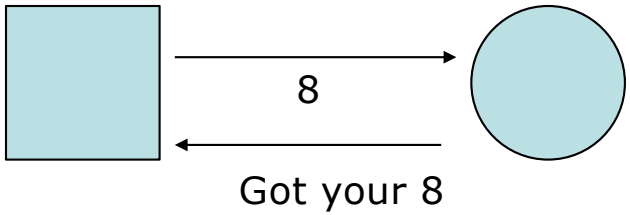
Quiz: How could signatures help?

Raft's primary focus is on reaching consensus in the presence of node or network failure.

→ Digital Signatures

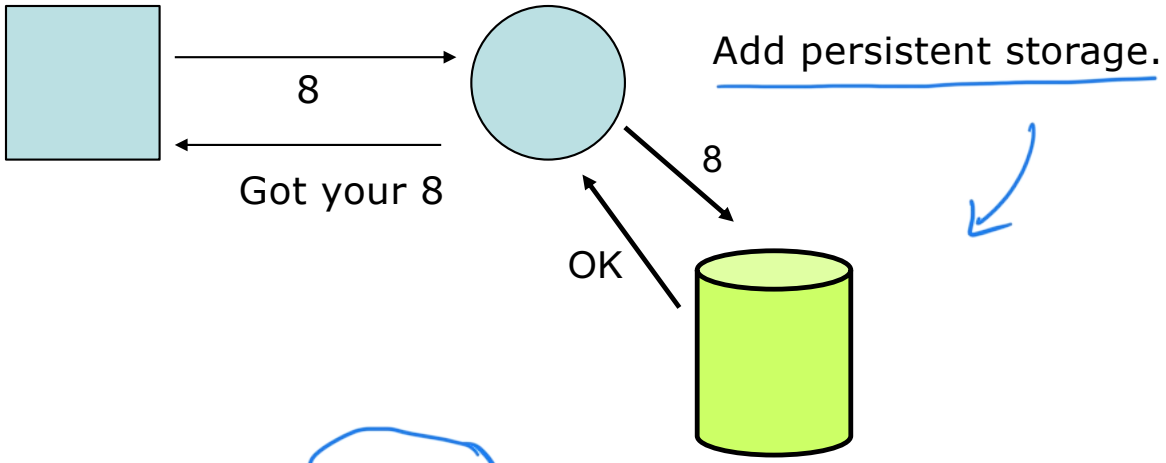
Reliability in the face of node failure

(Raft's concern)



Suppose the server crashes after responding.

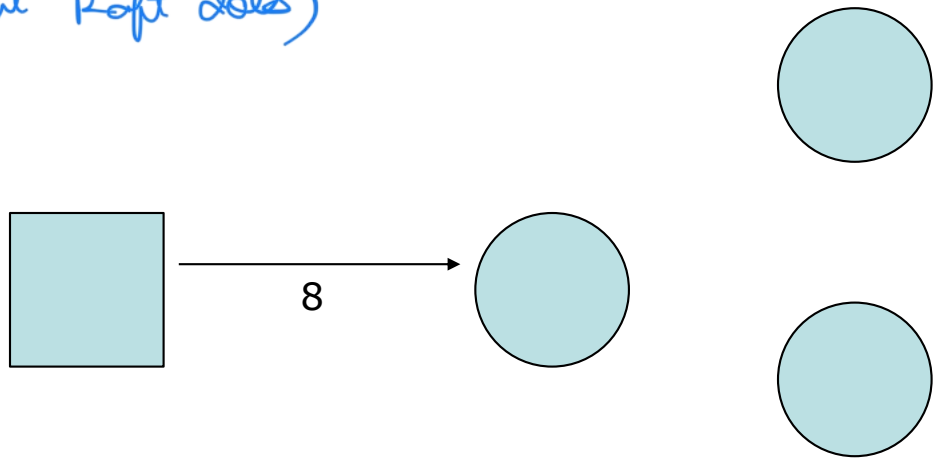
Reliability in the face of node or network failure



But what if the server and disk drive both crash after responding?
We need more computers.

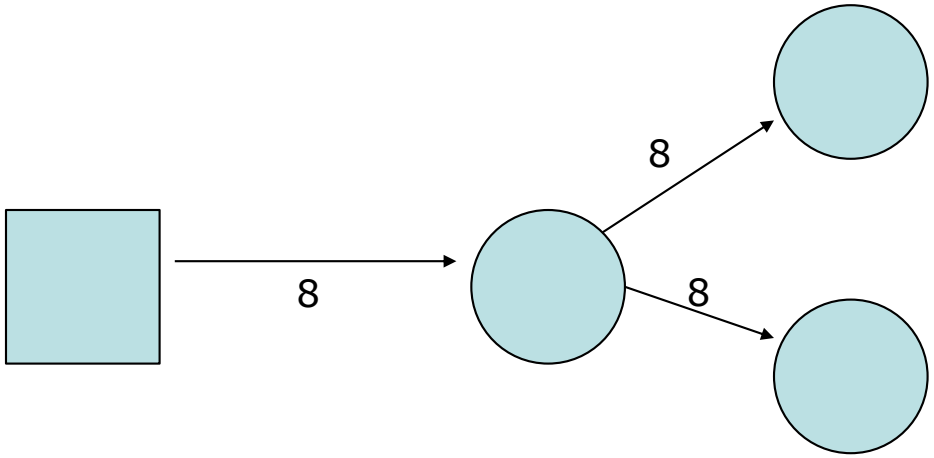
Improving Reliability

(This is what Raft does)

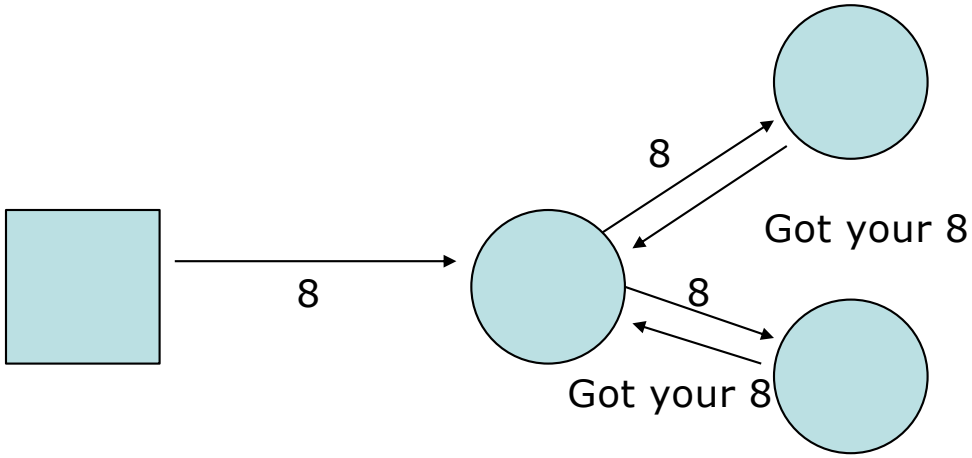


More computers (servers)

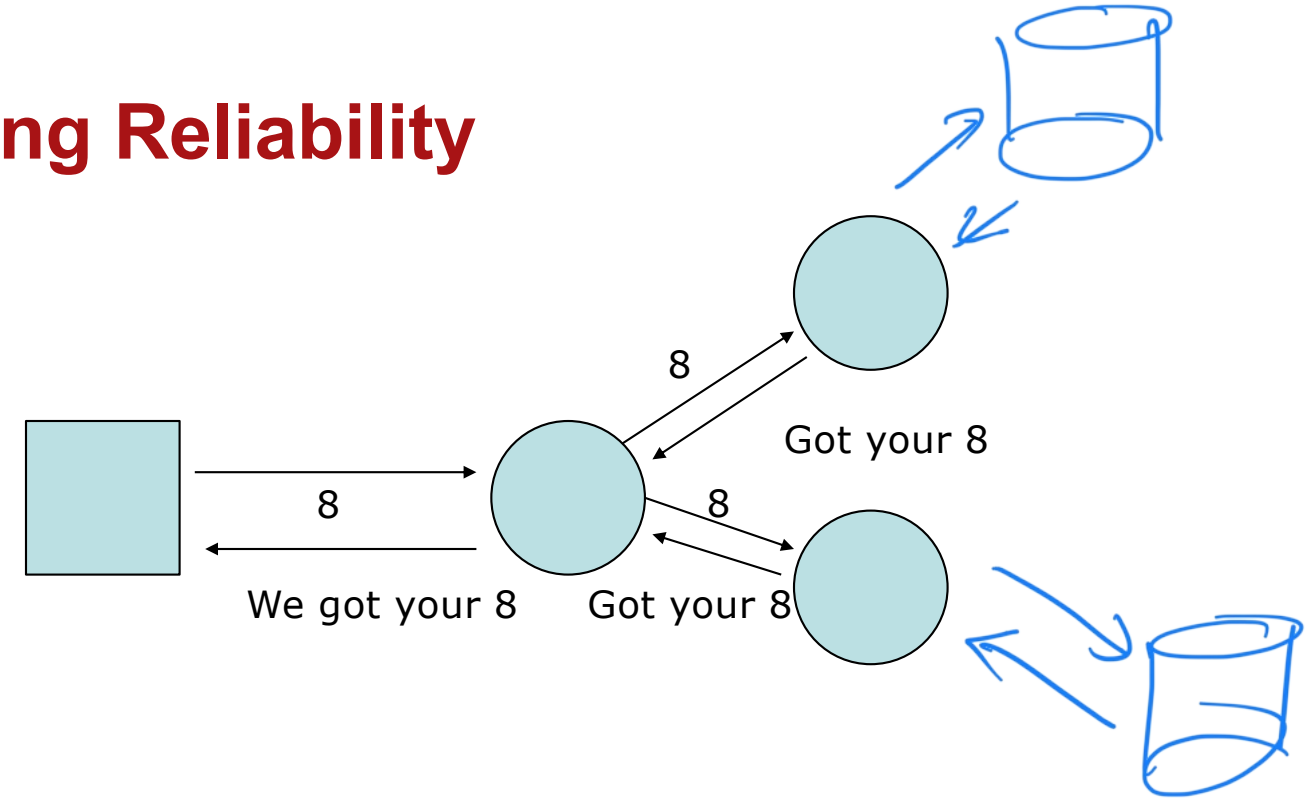
Improving Reliability



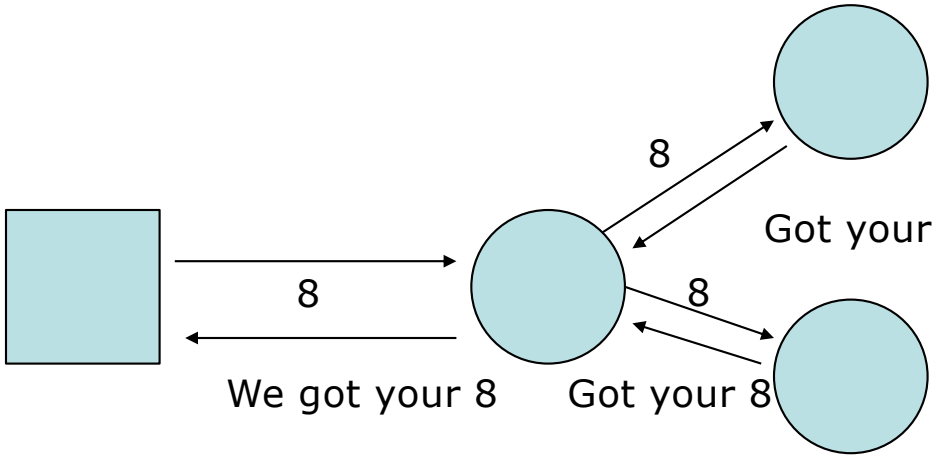
Improving Reliability



Improving Reliability



Improving Reliability



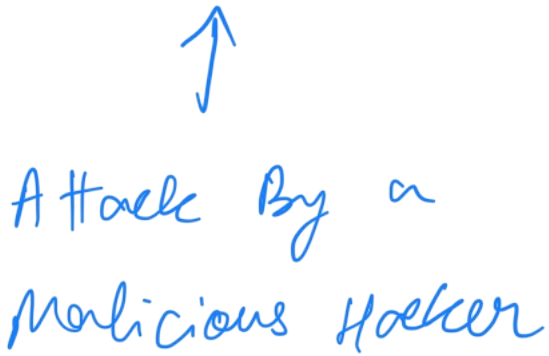
All requests go through the leader.
What if the leader dies?
How do we detect that?
How do we get all to agree on the next leader?
What if there is a partition between the servers?
Raft is designed to answer these concerns.

Raft Handles

Network Failure

Raft animation: Understanding Raft

Unlike Raft, Nakamoto Consensus in bitcoin is designed to be Byzantine Fault Tolerant



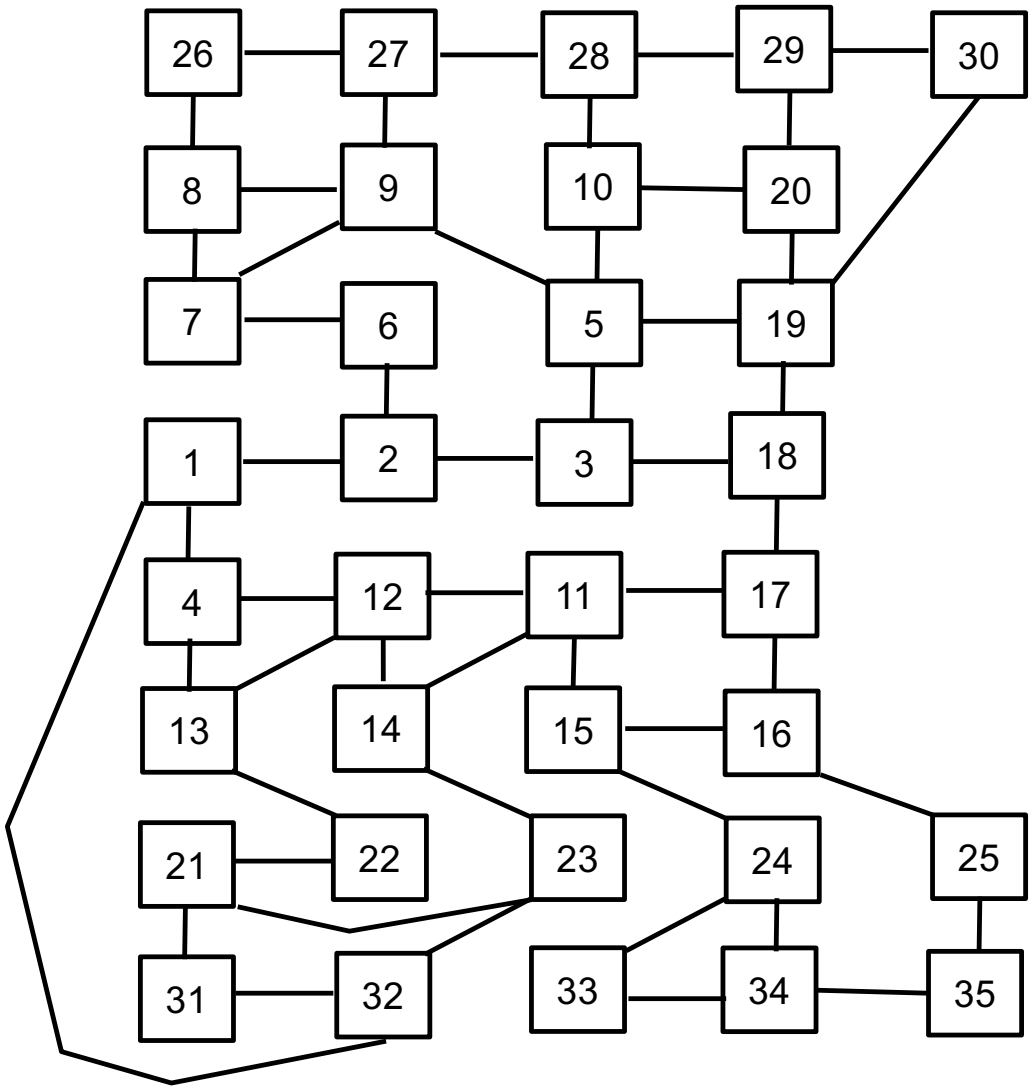
From the Bitcoin paper by Satoshi Nakamoto

6 steps :

1. New transactions are broadcast to all nodes.
2. Each node collects new transactions into a block.
3. Each node works on finding a difficult proof-of-work for its block.
4. When a node finds a proof-of-work, it broadcasts the block to all nodes.
5. Nodes accept the block only if all transactions in it are valid and not already spent.
6. Nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash.

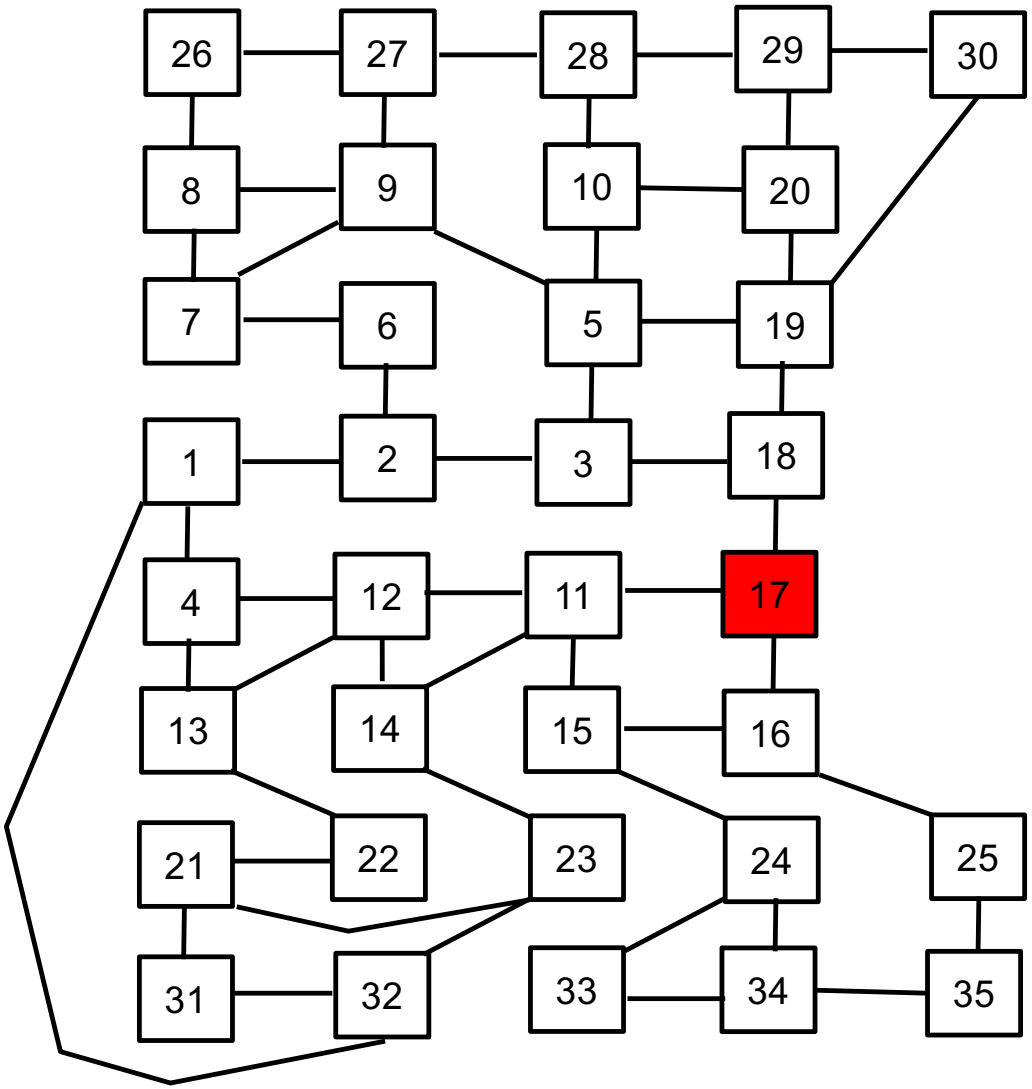
Let's visit each of these steps:

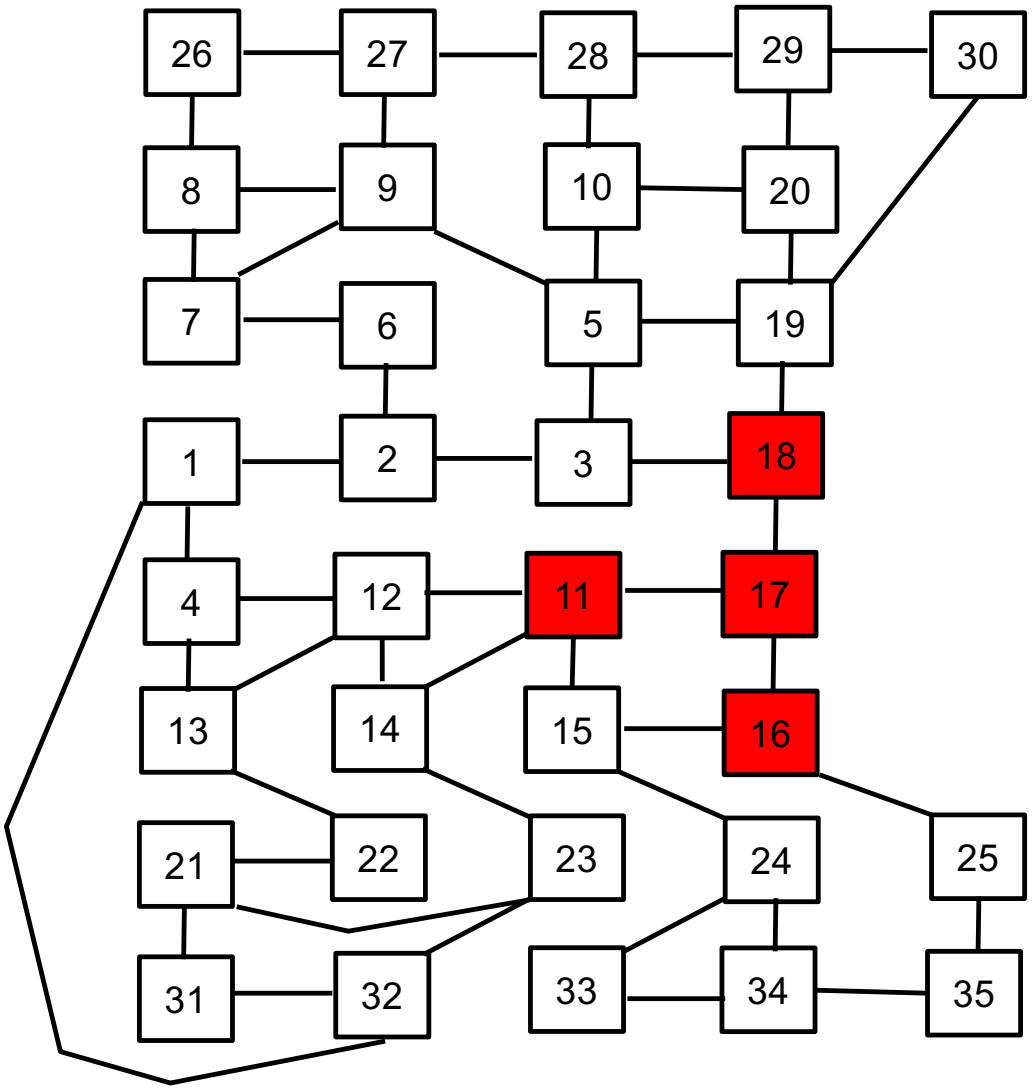
1. New transactions are broadcast to all nodes.

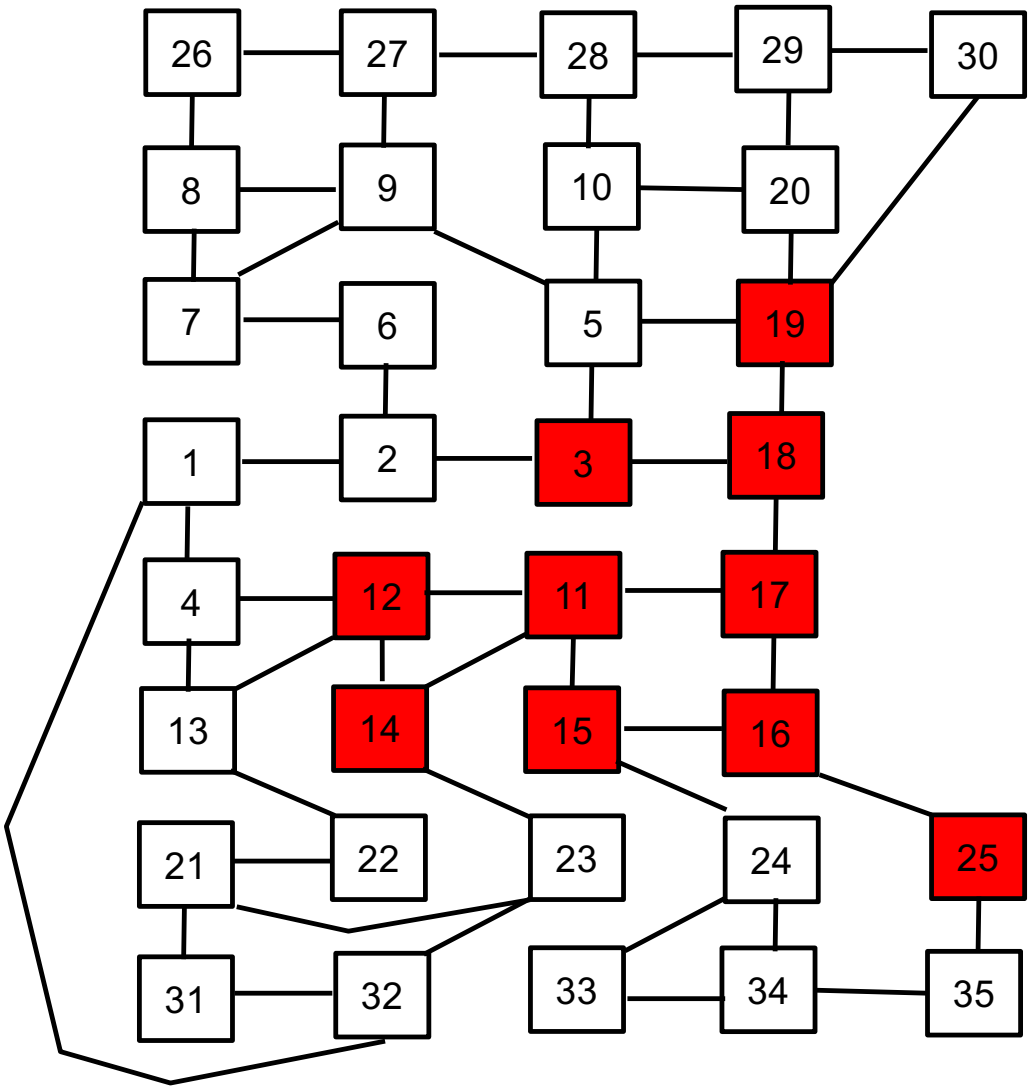


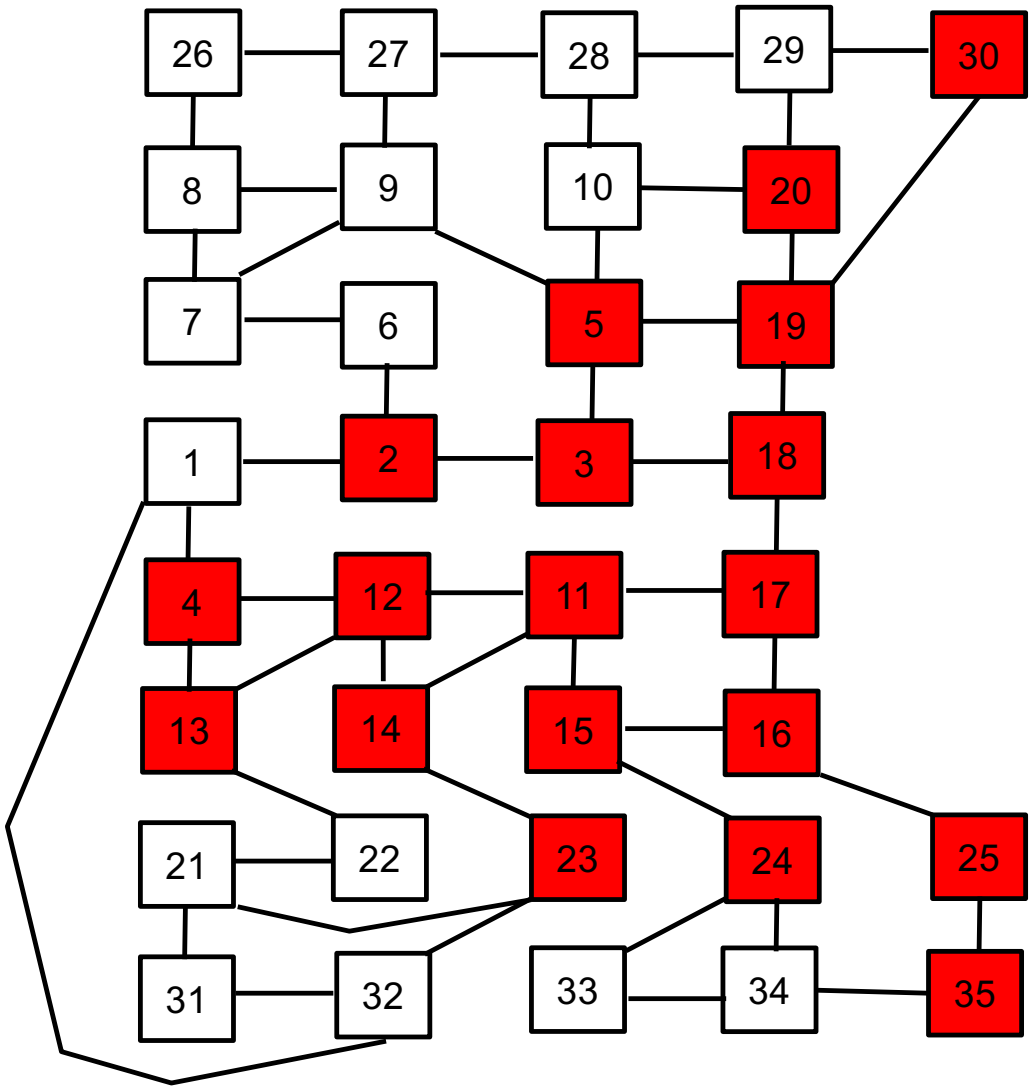
Peer to peer distributed system
No central authority
Nodes act as clients and servers
Network is asynchronous
Accurate time keeping by all nodes is not possible
Nodes communicate only by passing messages
We may lose messages
Each node knows only about a small set of others
Easy to join and leave without disruption
No single point of failure
Fault tolerant
Messages may be flooded though the network
Flooding is a simple gossip or epidemic protocol

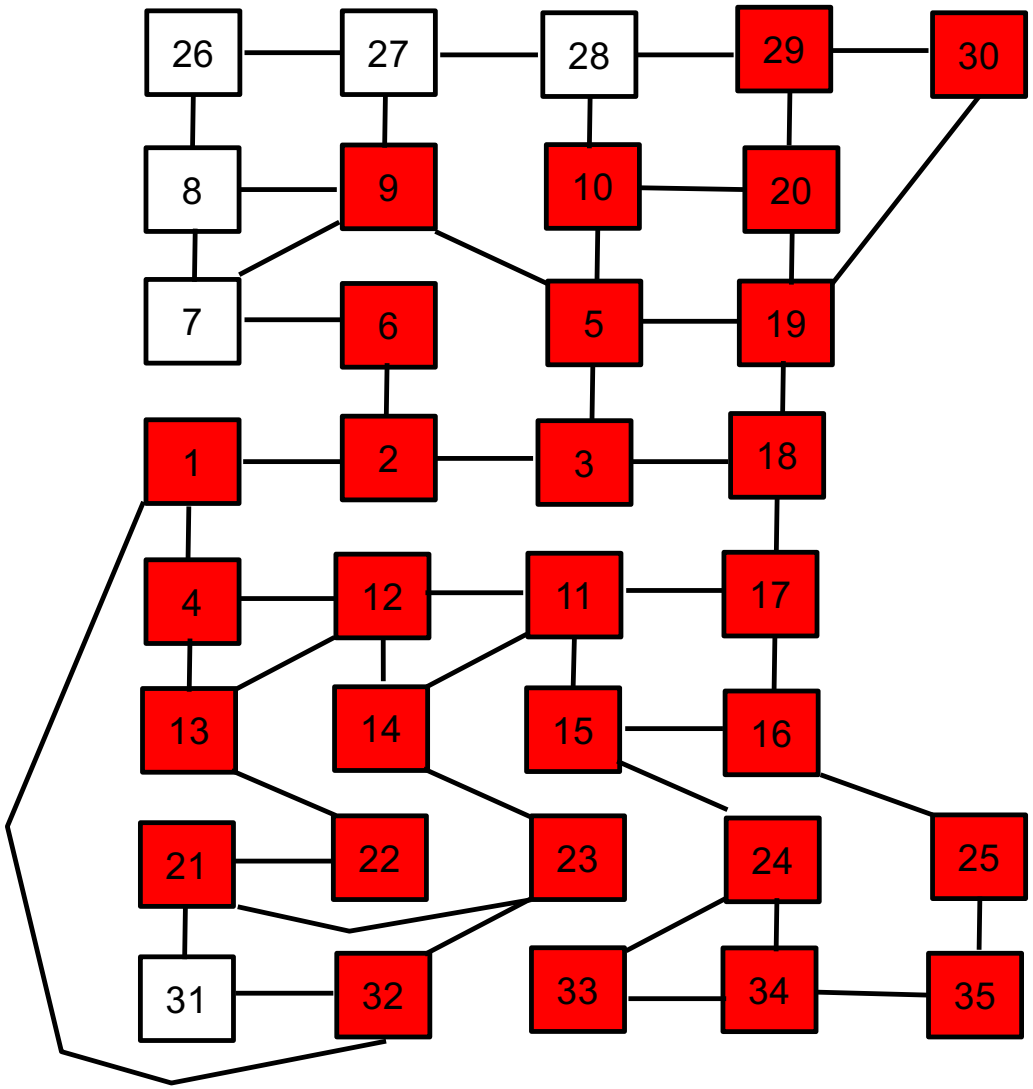
Suppose we provide a message to node 17
Node 17 will spread the gossip among its peers

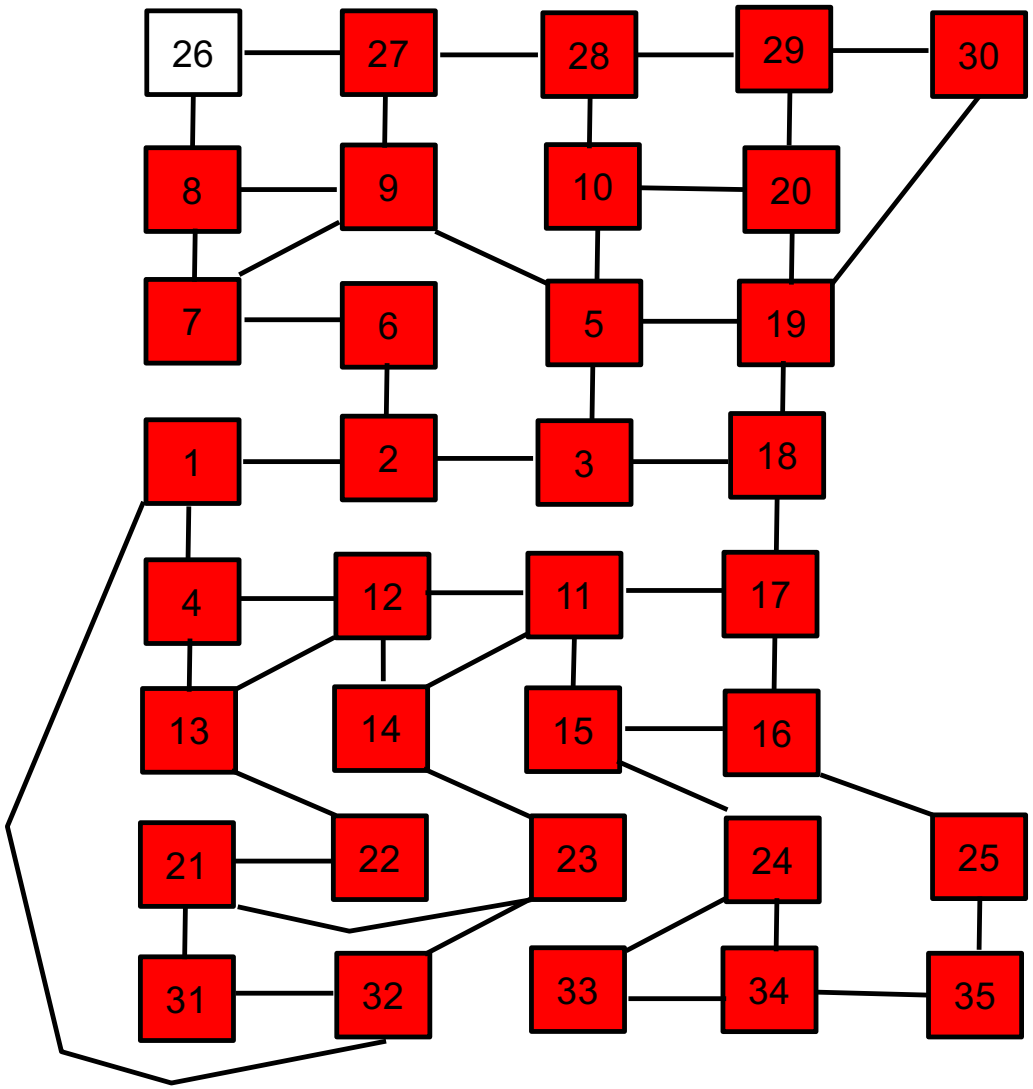

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

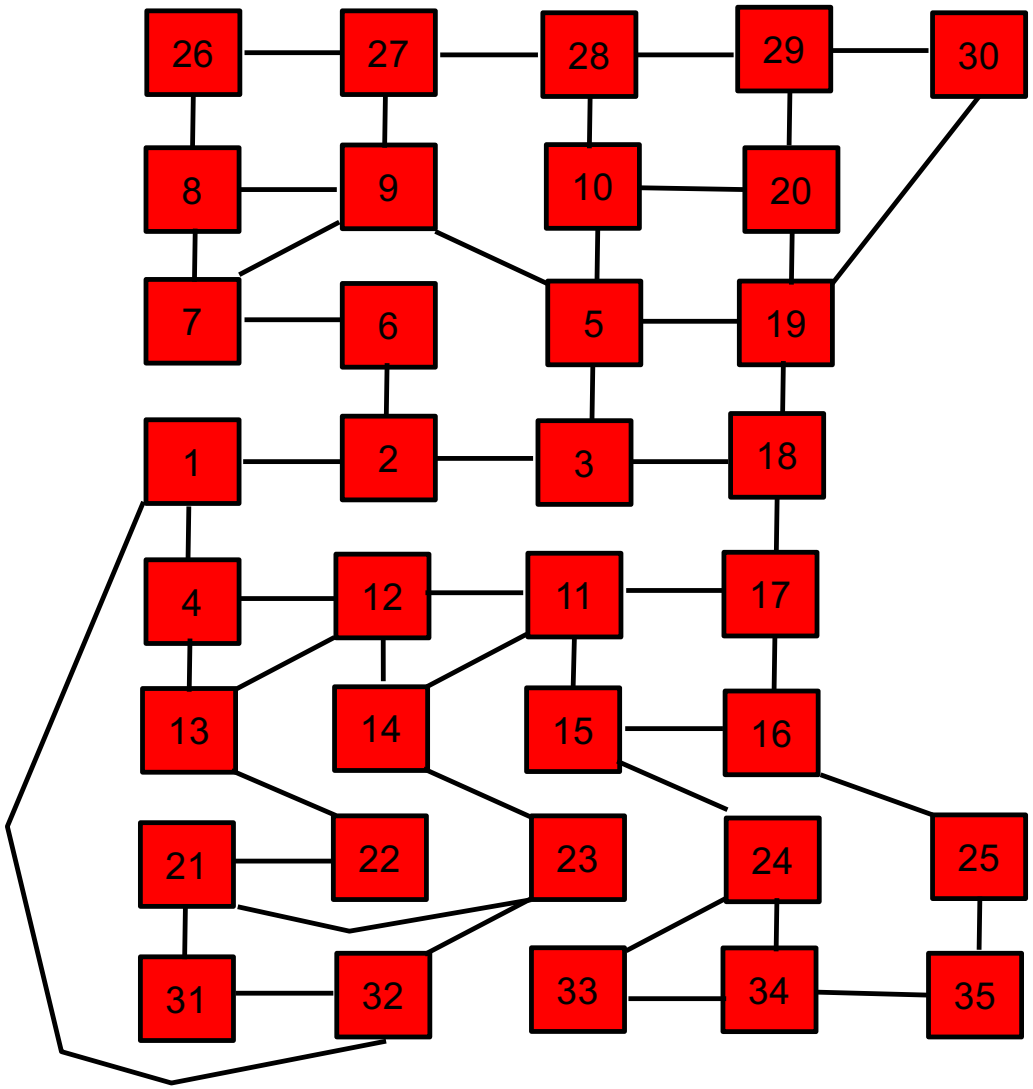

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

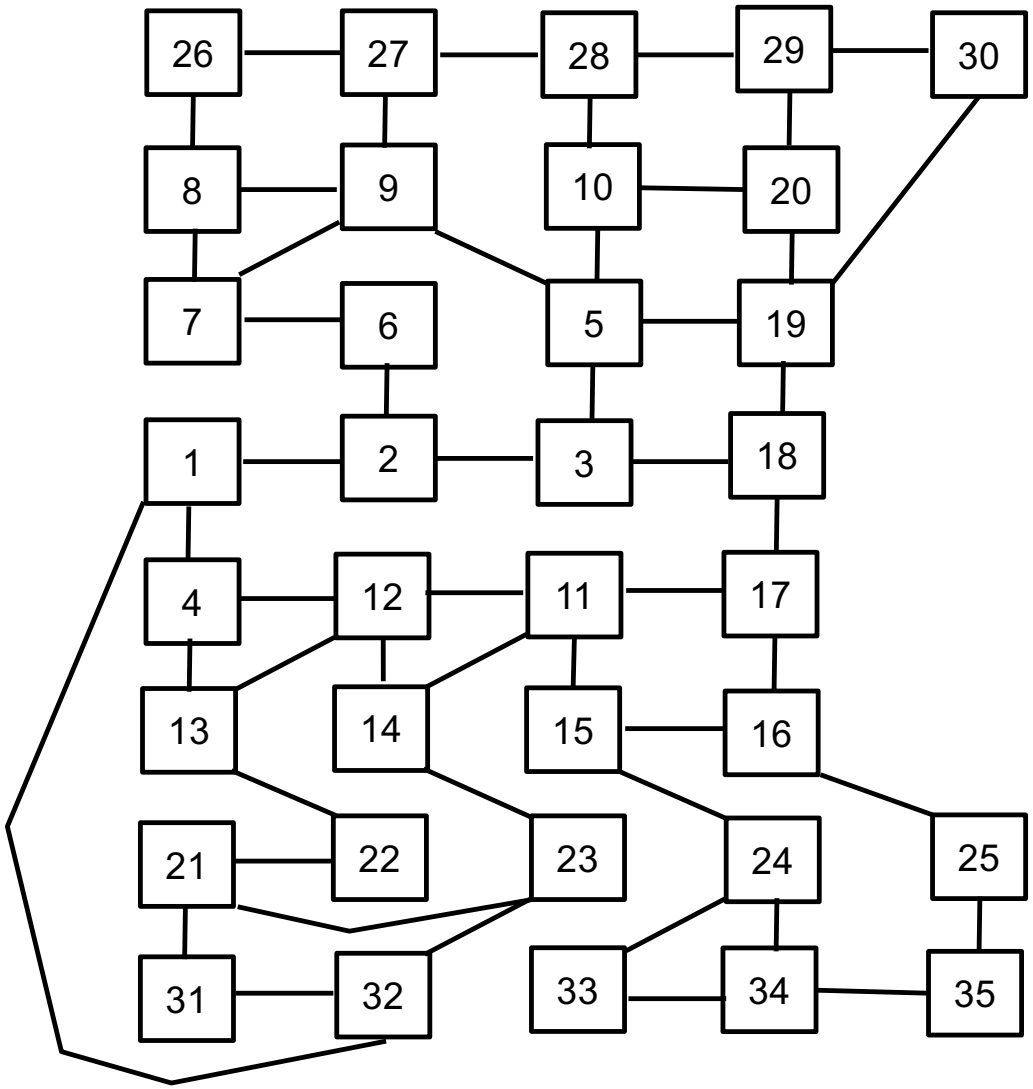

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

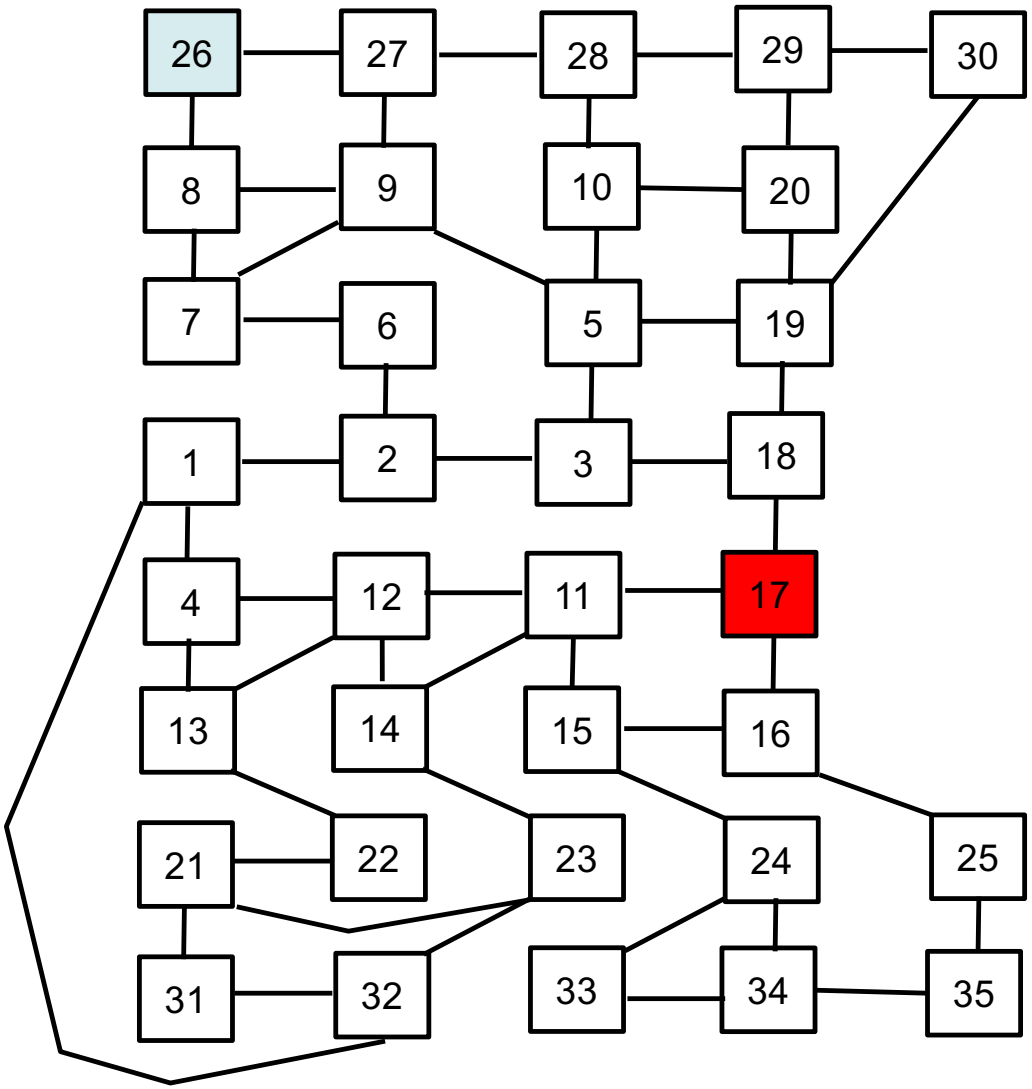


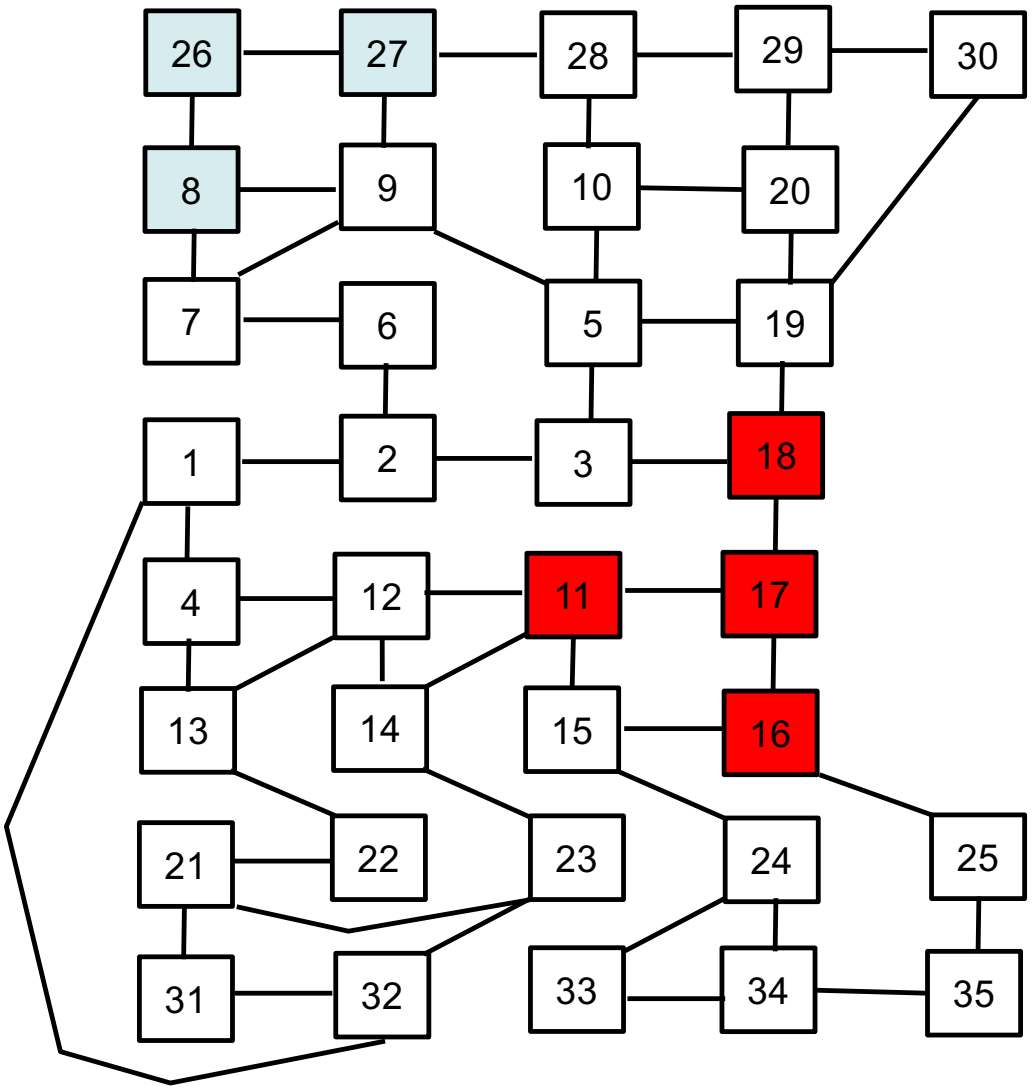
How do we know when to stop?
Don’t gossip a message to anyone that you know already heard it.

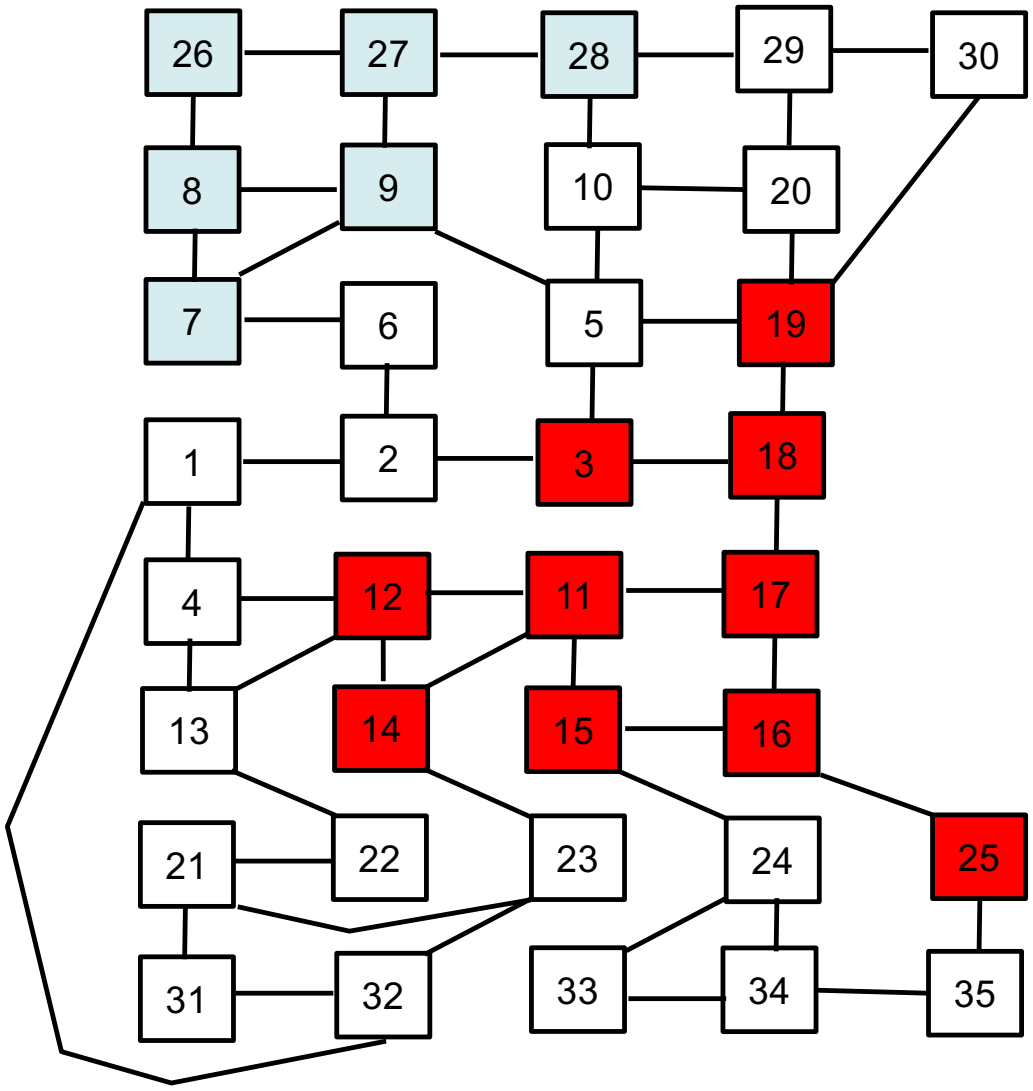
2. Each node collects new transactions into a block.



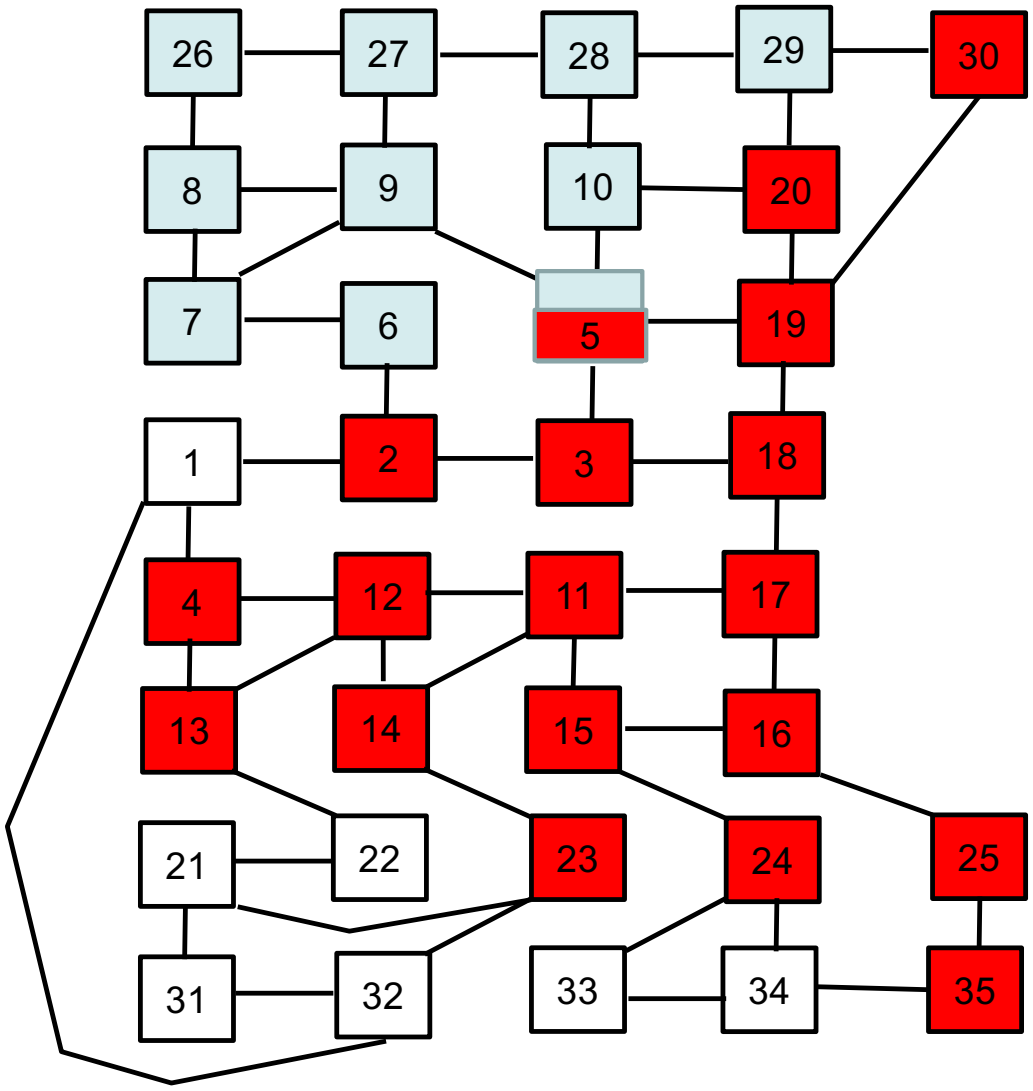
Gossiping a single message is easy.
But suppose we have two messages entered into the network at about the same time
Node 17 gets one message and another to node 26


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

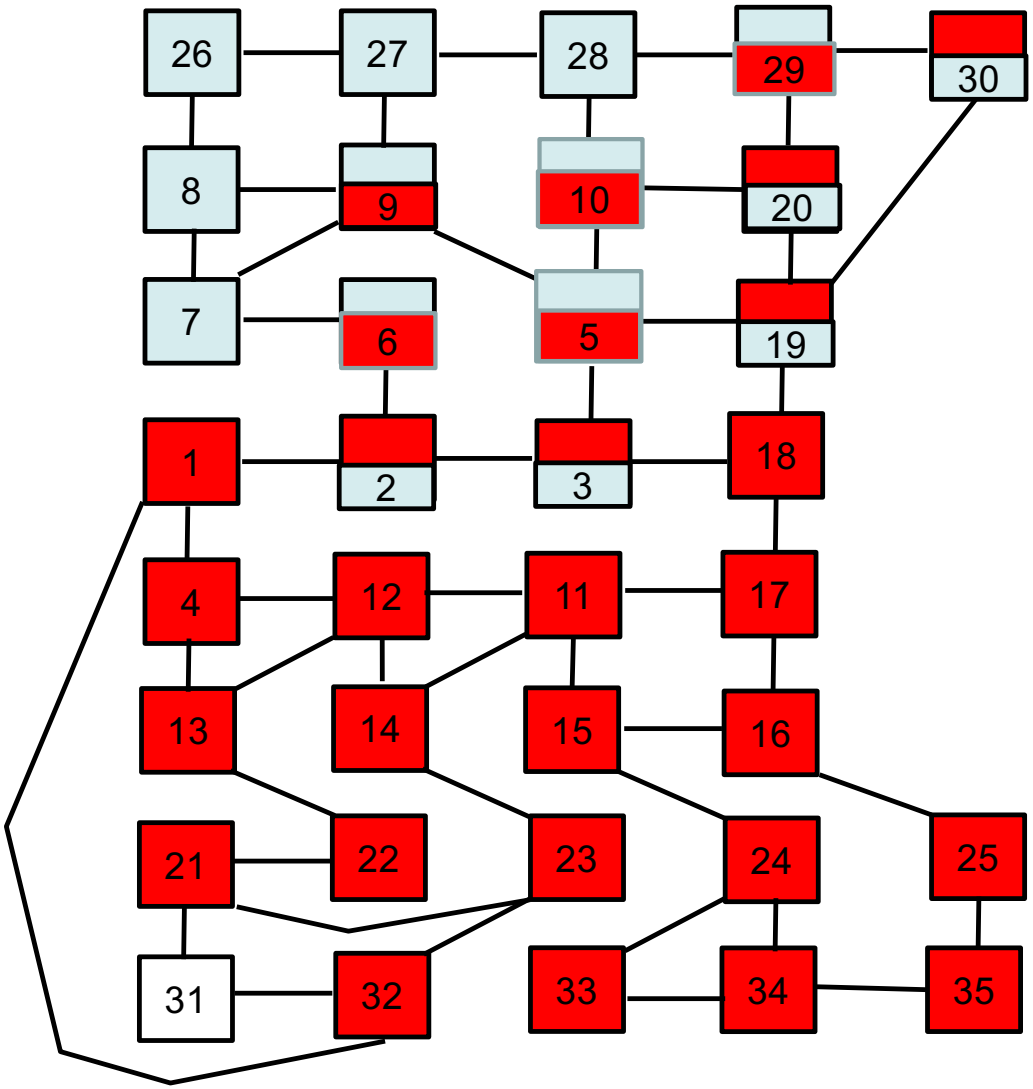

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

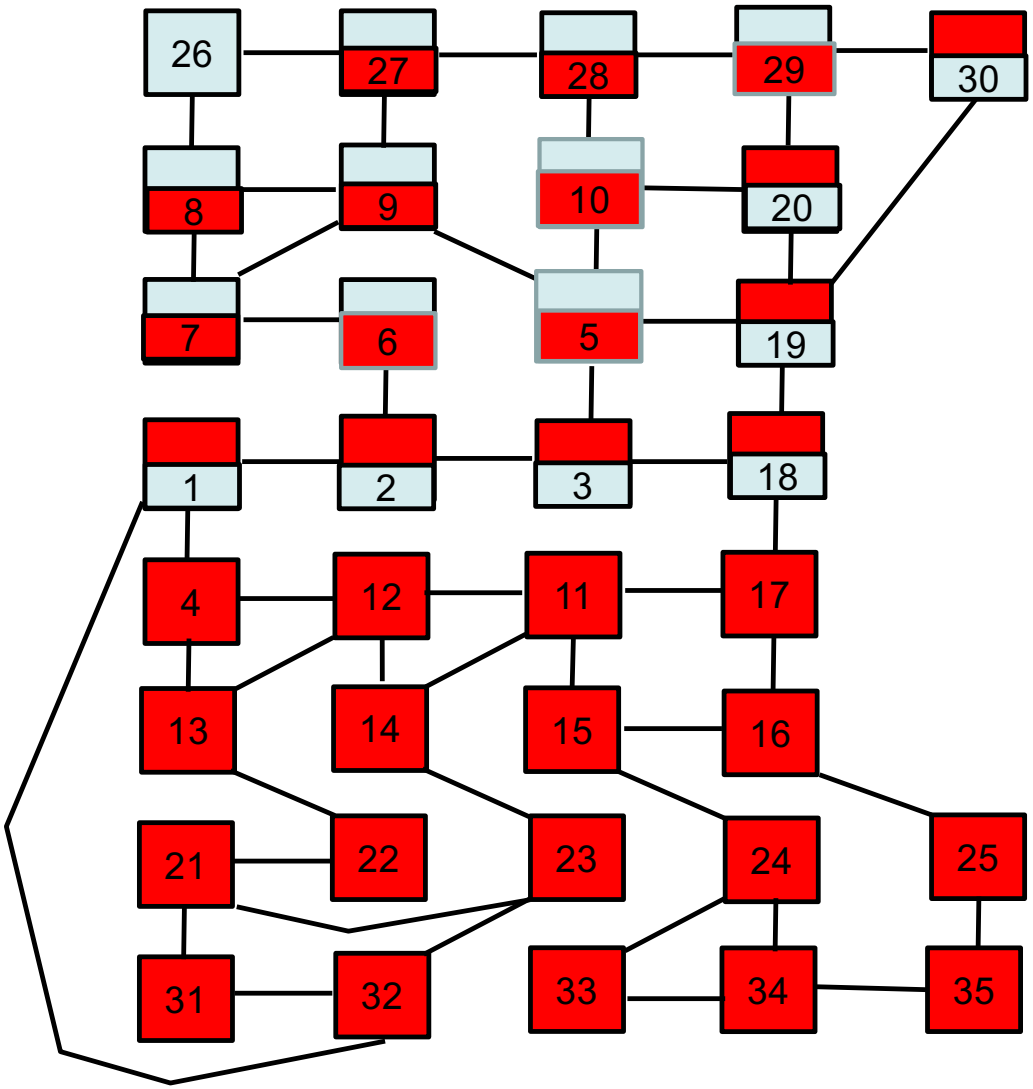


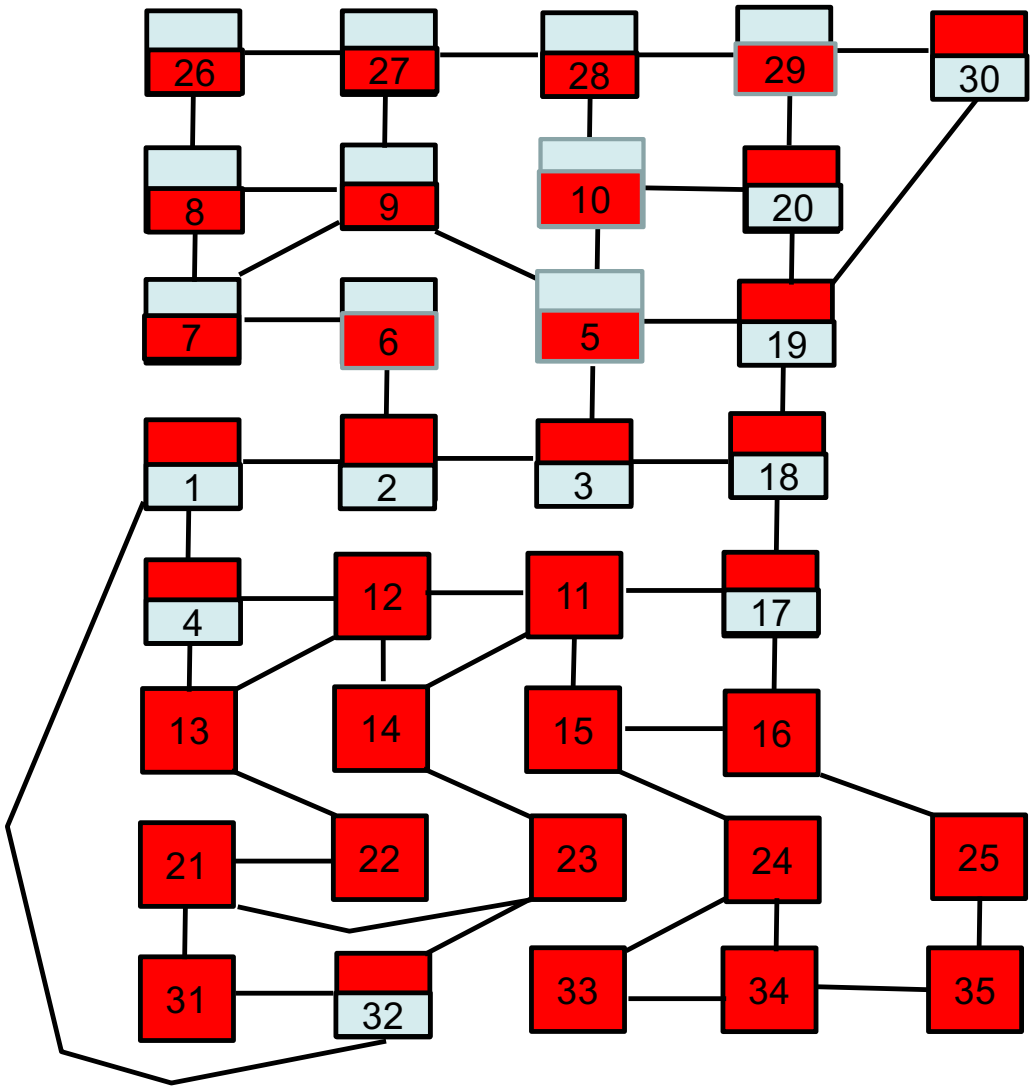
Node 5 is just about to hear two messages
One will be blue and the other red
Node 5 might receive blue first or red first
It needs to keep both messages

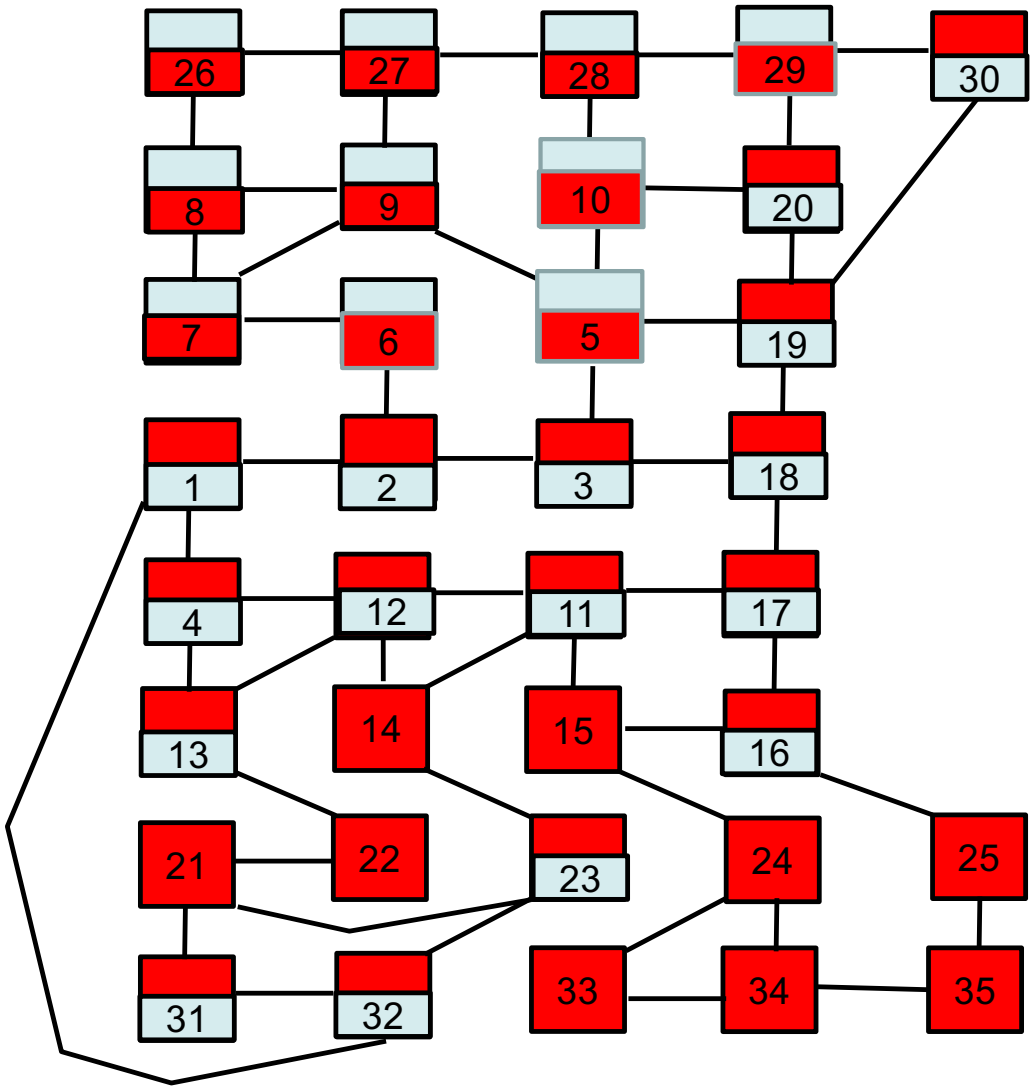


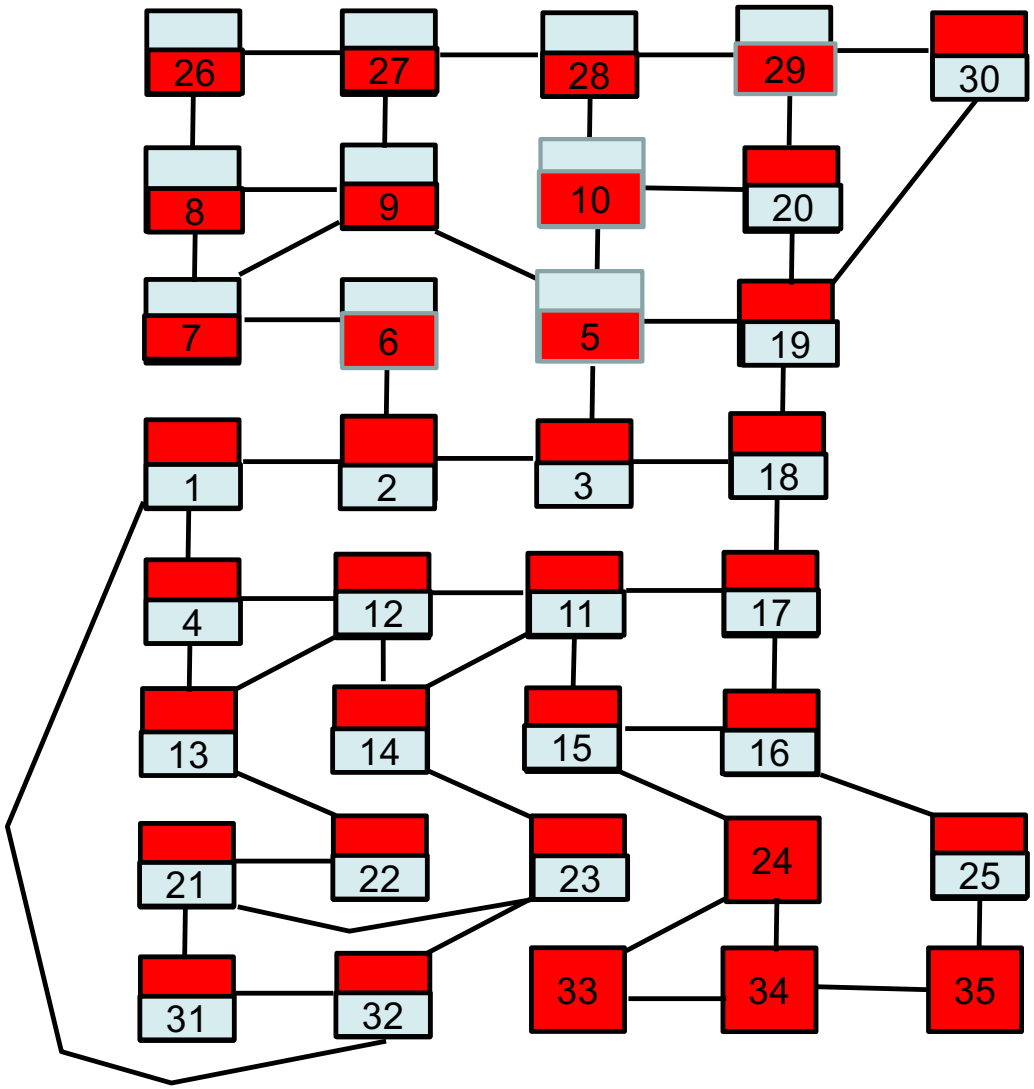
Node 6 is currently blue
When the red message arrives it needs to keep both
It will keep blue as the first message and red the second

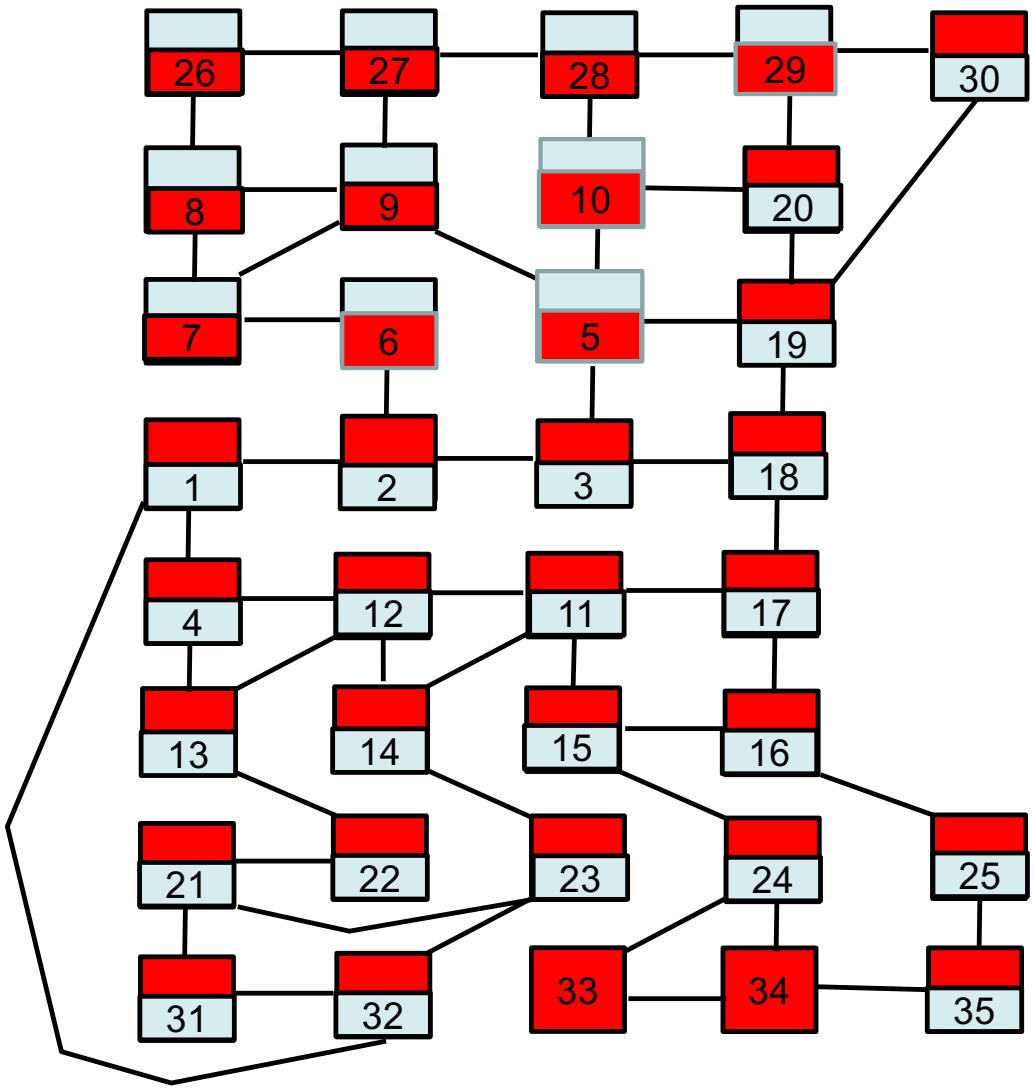

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

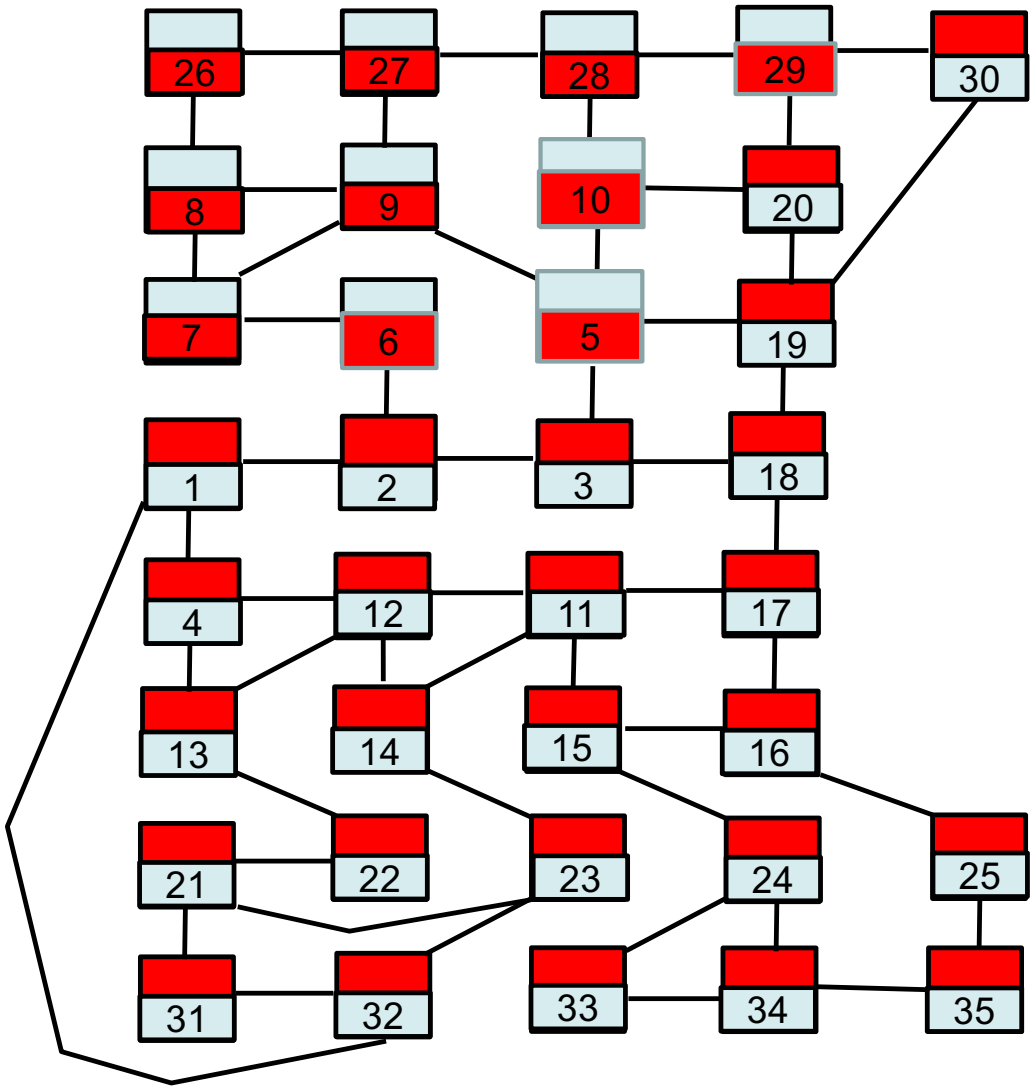

26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


26
27
28
29
30
8
9
10
20
7
6
5
19
1
2
3
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35



All of the peers now know both messages

But we lack consensus on the order of the messages. Some have red on top and some have blue on top

We need every node to have the exact same set of messages – in the same order

How can we resolve this?

Proposal: Choose a node at random and gossip its ordering

Everyone else adopts that ordering

That approach would work but who is choosing the node at random? Who is rolling the dice? We do not want any central authority

Solution: Require that each node perform an activity that requires time and effort. The activity is competitive in that the first to arrive at an agreed upon value wins!

It gossips the ordering

But how do we know that the peer has not simply pretended to do the activity?

We need each node to verify that the activity was performed.

3. Each node works on finding a difficult proof-of-work for its block.

We need some tools from cryptography

- We need SHA256 and proof of work
- Suppose h = SHA256(“Some arbitrary message”);
- h is called a cryptographic hash of the message
- h is 32 bytes and each bit is hard to predict without running SHA256
- h is always the same if the message is the same
- If all we know is h, we cannot learn the message
- h is not the message encrypted
- There are 2^256 different SHA256 hashes. This is far, far more than the number of atoms in our solar system.

We need some tools from cryptography

- Proof of work (be sure to trace this code)
- Consider a difficulty of 2:

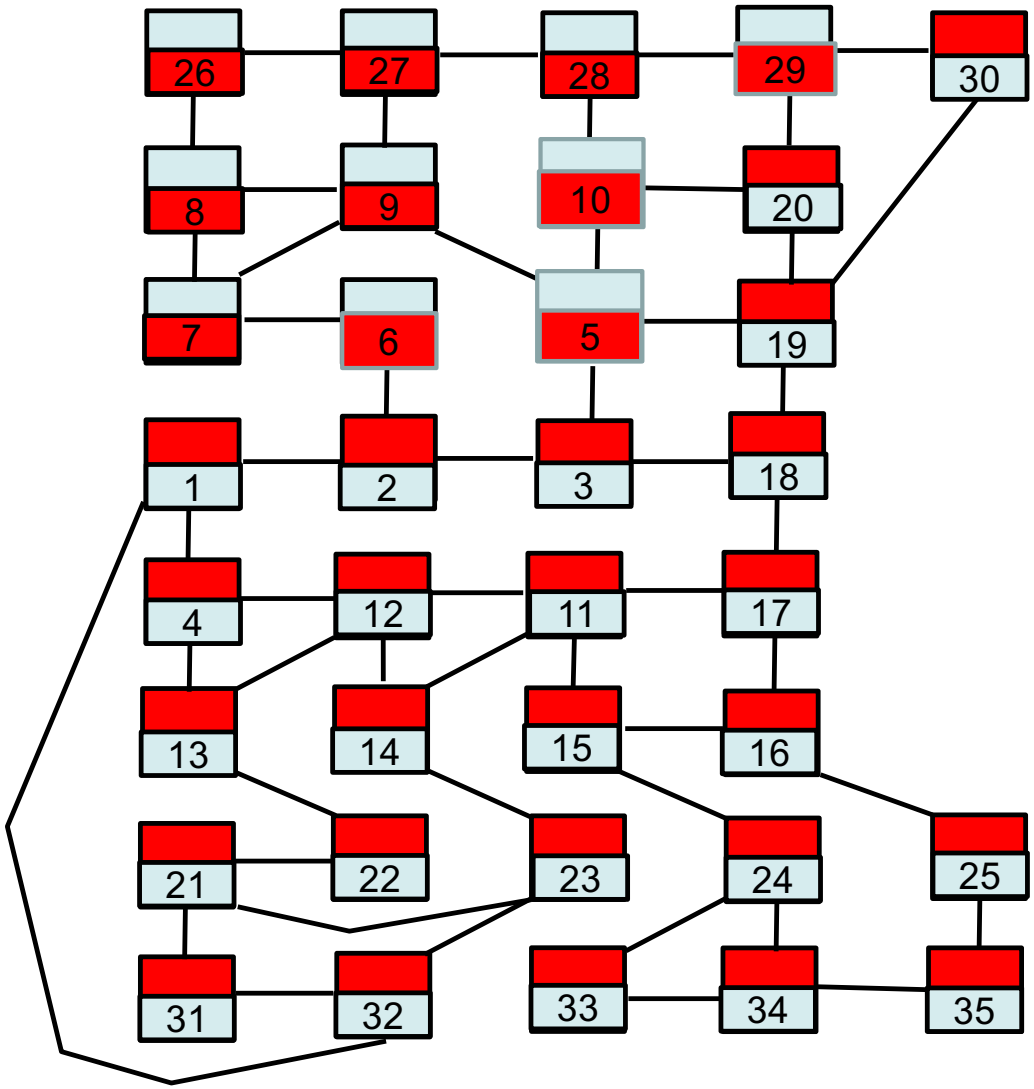
```
int nonce = 0;
 h = SHA-256(someData + nonce);
while(two leftmost hex digits of h are not 00) {
    nonce = nonce + 1;
      h = SHA-256(someData + nonce);
}
```

- Trace the execution
- On average, how many times will this loop execute?
- Prob("00") = 1/16 * 1/16 = 1/256, 256 trials expected

We need some tools from cryptography

- Proof of work
- Hard to compute proof of work but easy (one hash) to verify
- The difficulty can change. We can force this to take 2 weeks for a single machine
- This is 10 minutes for the peer to peer network as a whole.
- With one block being produced every 10 minutes, there will be a time gap between blocks.
- If computers become faster, we simply increase the difficulty

```
int nonce = 0;
h = SHA-256(someData + nonce);
while(two leftmost hex digits of h are not 00) {
        nonce = nonce + 1;
          h = SHA-256(someData + nonce);
}
```



Suppose peer 30 begins running proof of work with the following data:
nonce
30
Red message
Blue message

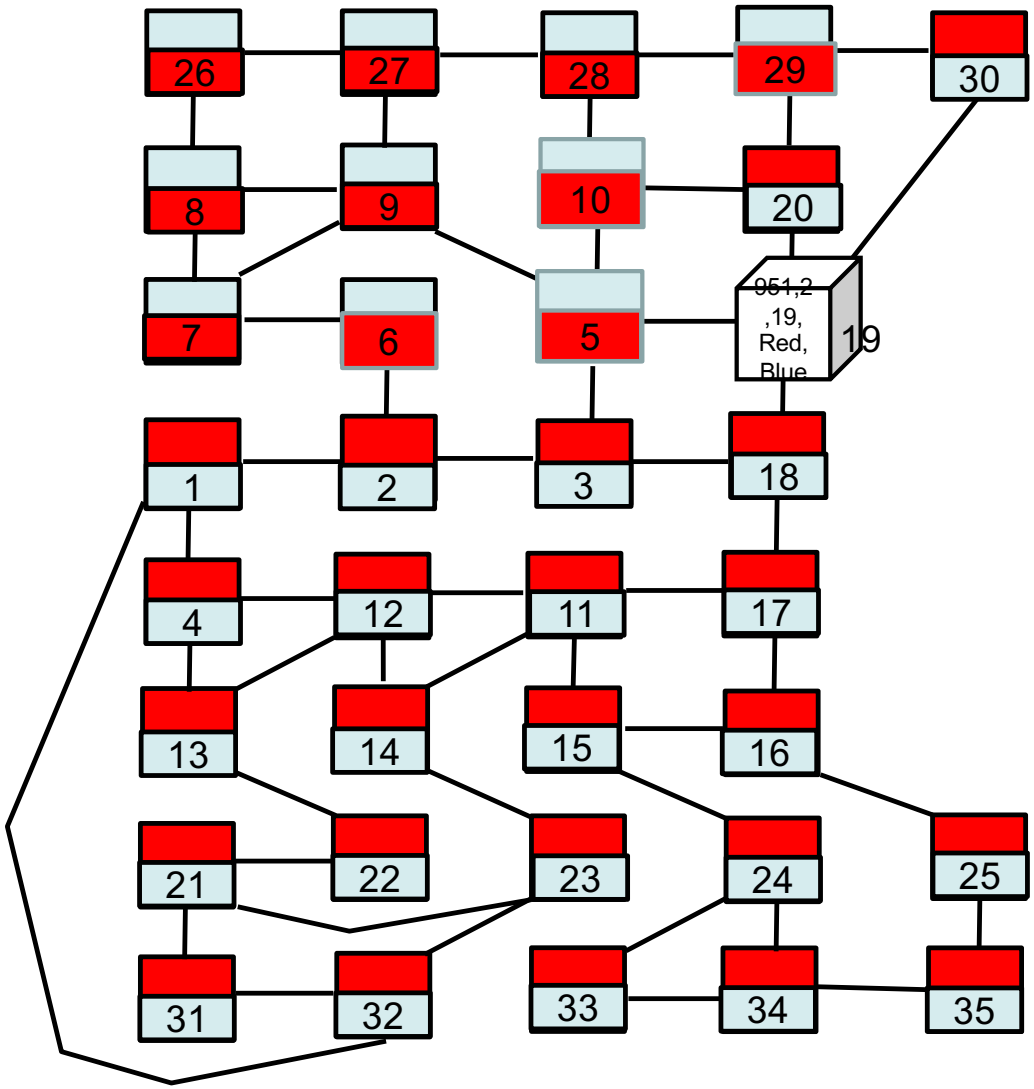
Suppose peer 5 begins running proof of work with the following data:
nonce
5
Blue message
Red message

Suppose everyone does the same.

If each SHA256 calculation takes 1 second, how long will it be before someone gets three or more initial hex 0's?

16*16*16 = 4096 expected hashes
4096 / 35 = 117 seconds on average

4. When a node finds a proof-of-work, it broadcasts the block to all nodes.



Suppose node 19 wins the race.

Node 19, worked with a difficulty of 2 to find the nonce of 951.

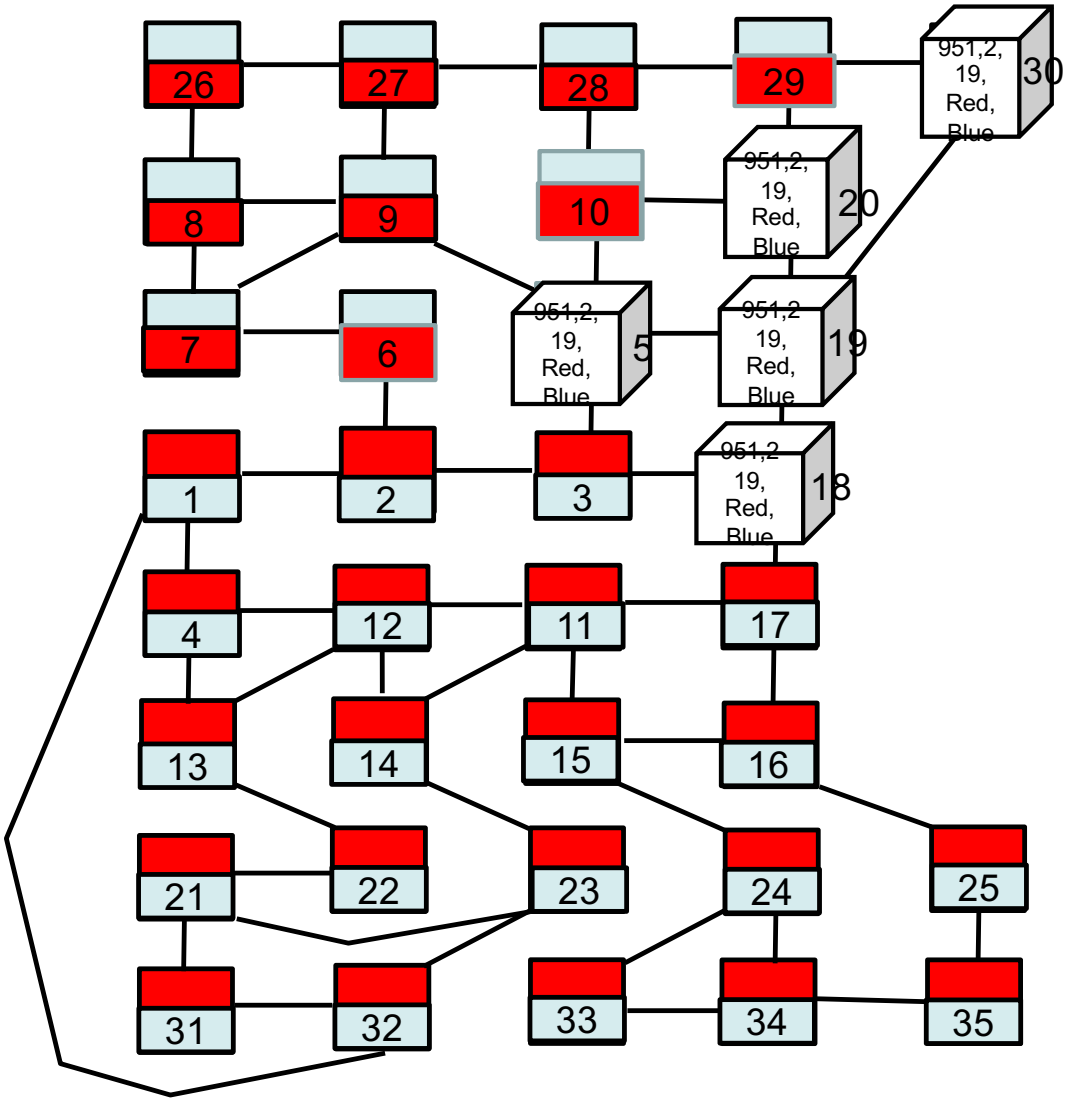
It found the hash of 951,2,19,Red,Blue is:

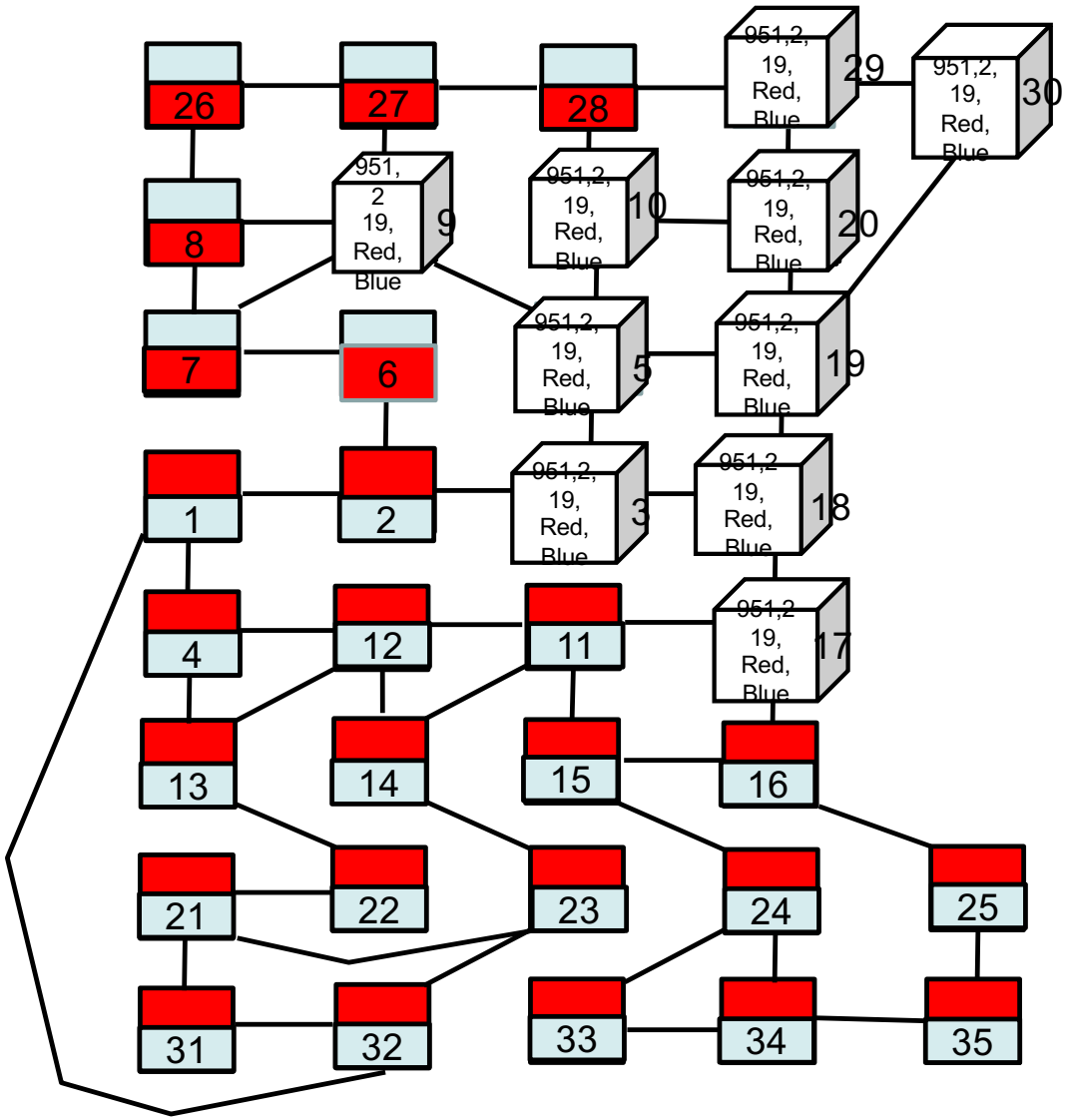
002fdb16086d97e03613fa0caa87b280eca956216e6
35400408bdd3a449e45

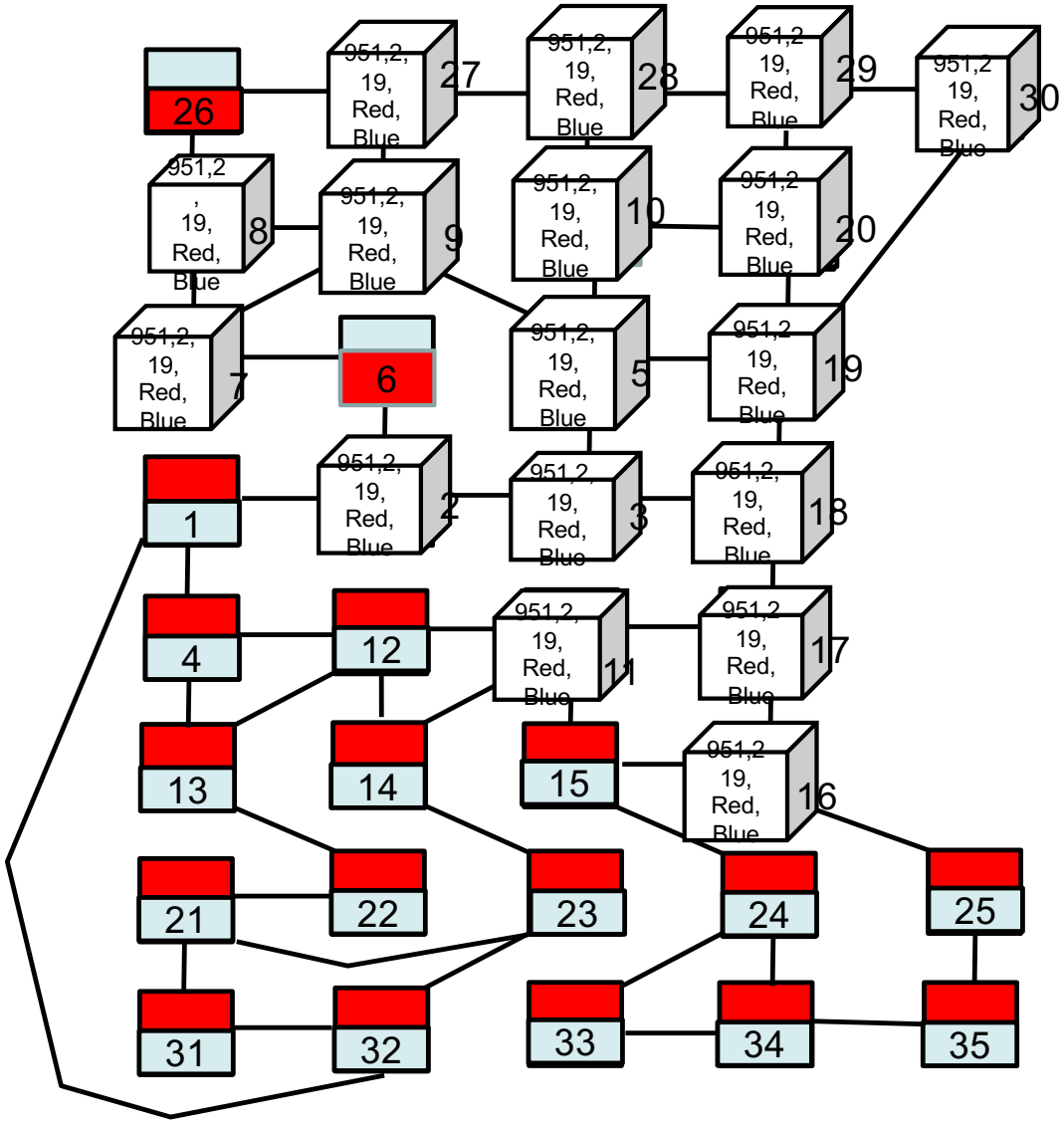
This hash is the name of this block.

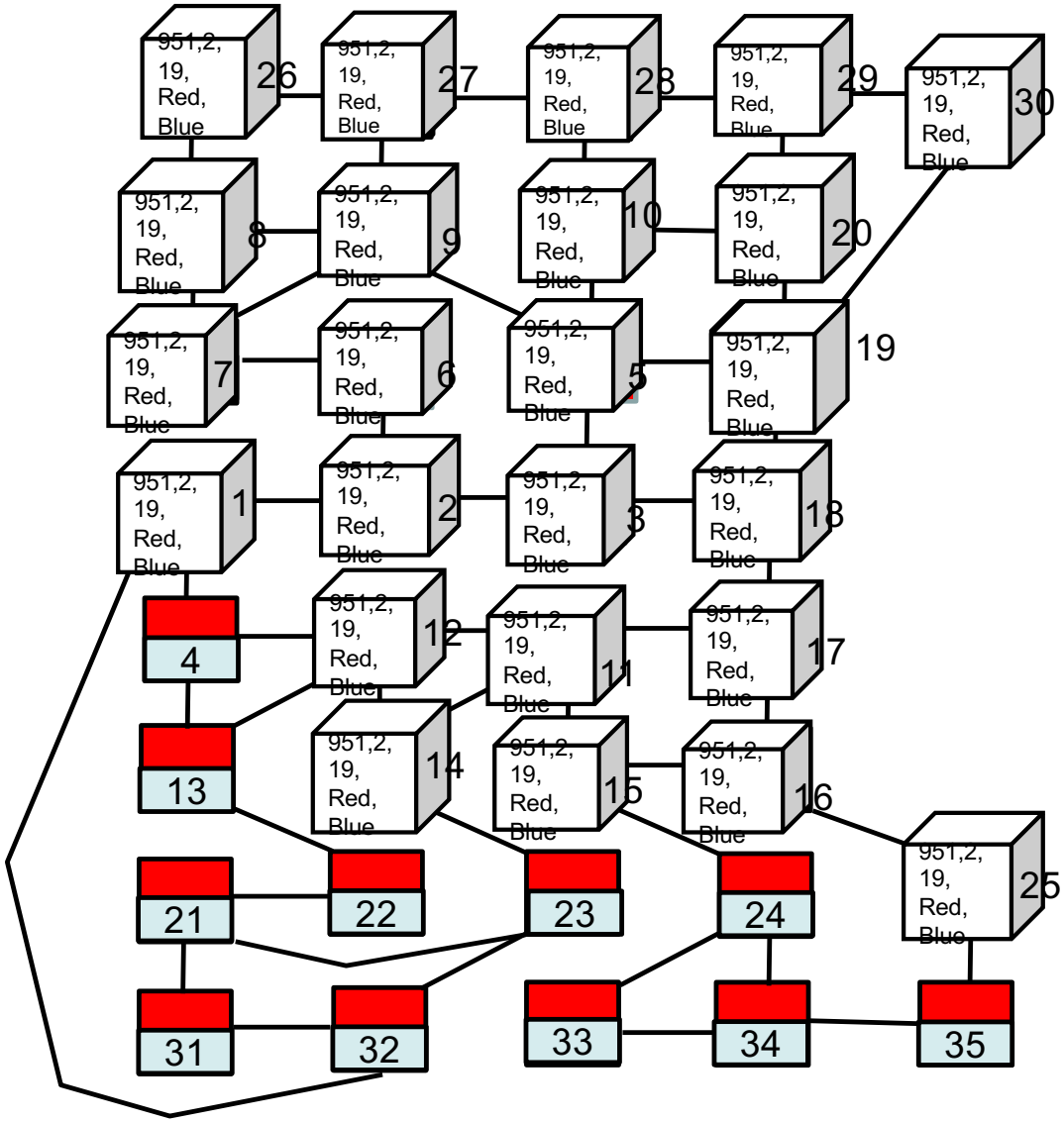
Node 19 will gossip the block.

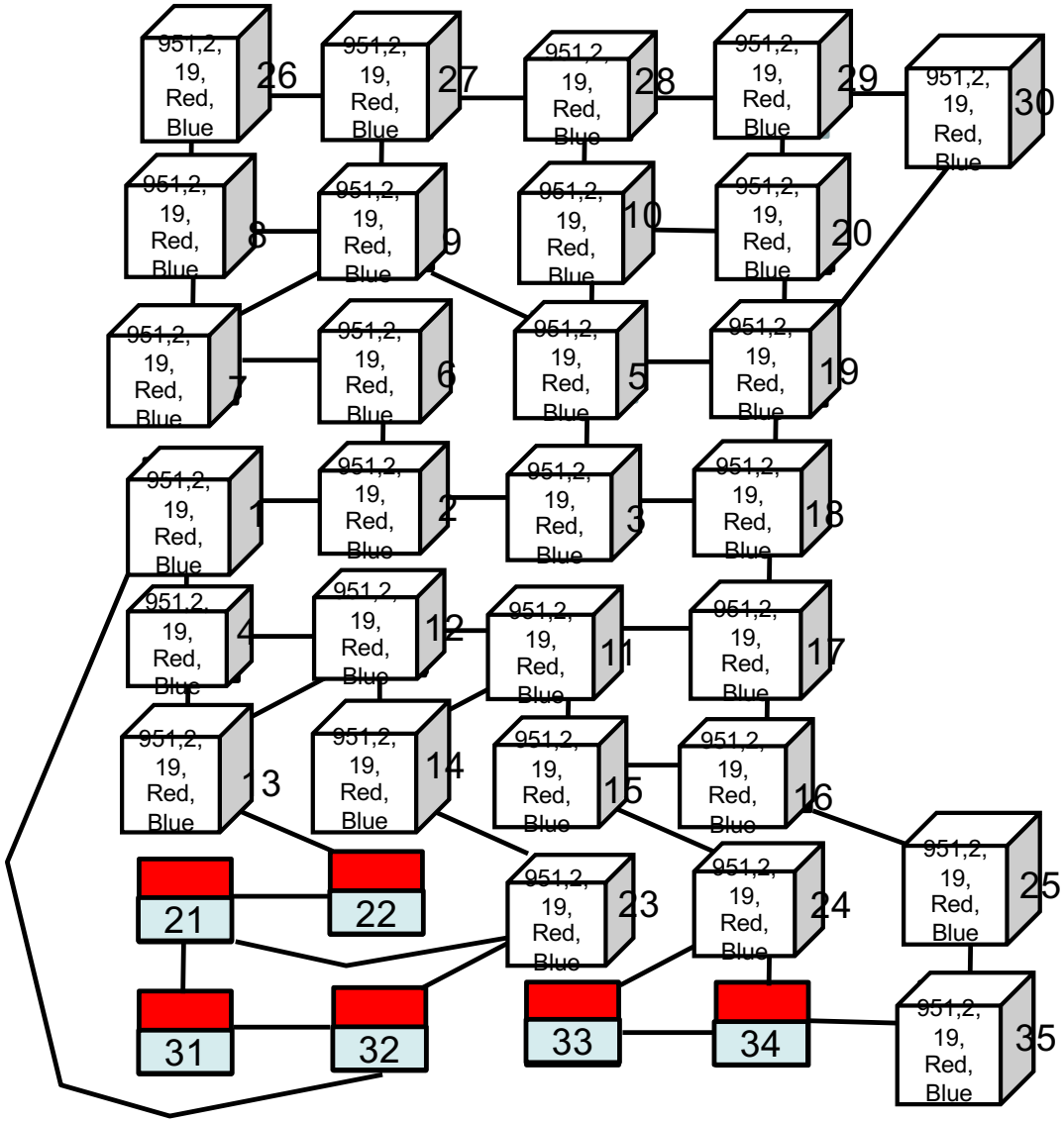
The block holds 951,2,19,Red,Blue

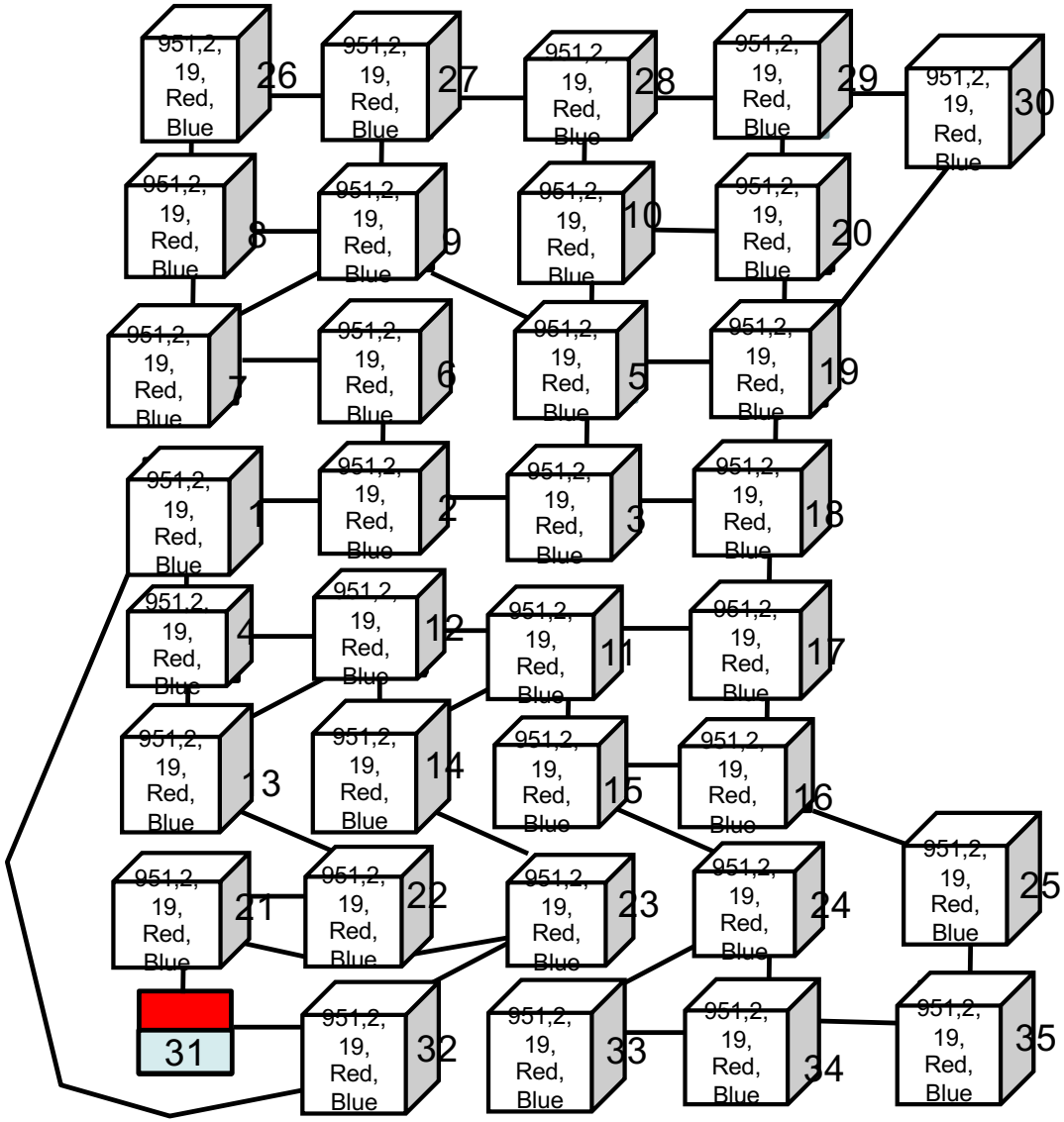

26
27
28
29
951,2,
19,
Red,
Blue
30
8
9
10
951,2,
19,
Red,
Blue
20
7
6
951,2,
19,
Red,
Blue
5
951,2
19,
Red,
Blue
19
1
2
3
951,2
19,
Red,
Blue
18
4
12
11
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35

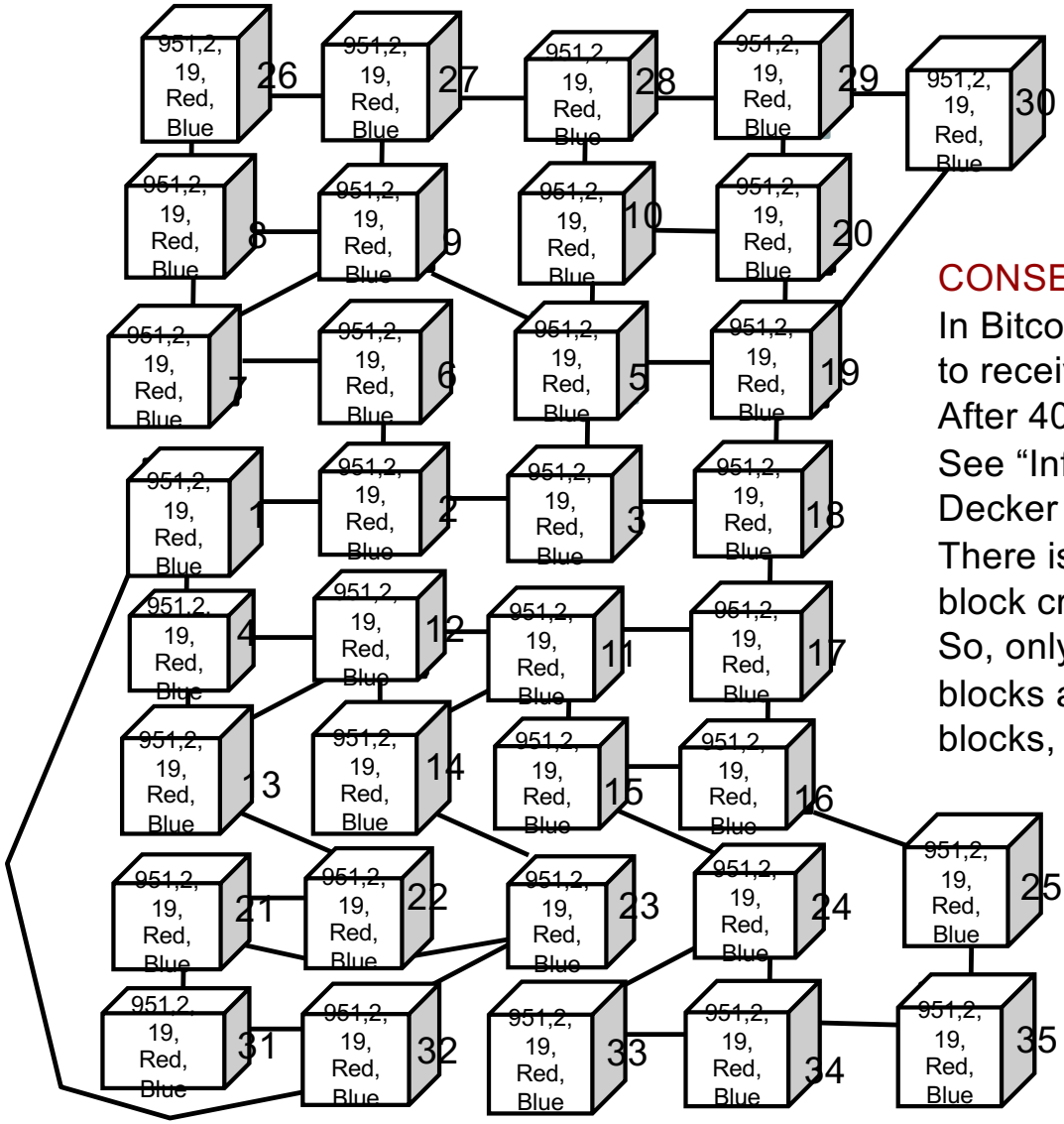

26
27
28
951,2,
19,
Red,
Blue
29
951,2,
19,
Red,
Blue
30
8
951,
2
19,
Red,
Blue
9
951,2,
19,
Red,
Blue
10
951,2,
19,
Red,
Blue
20
7
6
951,2,
19,
Red,
Blue
5
951,2,
19,
Red,
Blue
19
1
2
951,2,
19,
Red,
Blue
3
951,2
19,
Red,
Blue
18
4
12
11
951,2
19,
Red,
Blue
17
13
14
15
16
21
22
23
24
25
31
32
33
34
35


951,2,
19,
Red,
Blue
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35


951,2, 19, Red, Blue
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35


951,2, 19, Red, Blue
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35


951,2, 19, Red, Blue
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35



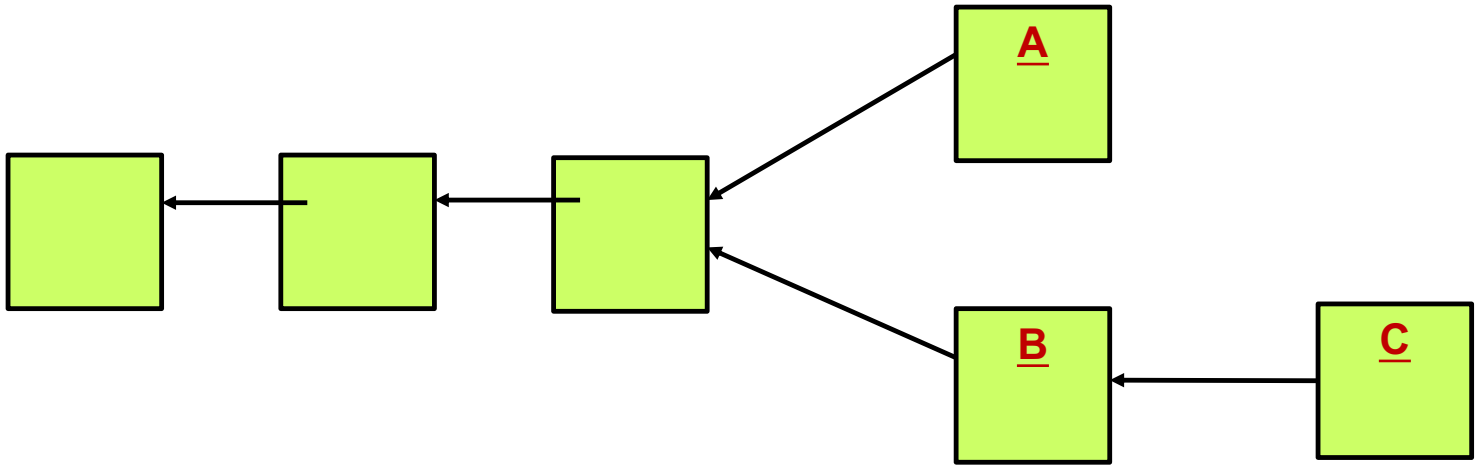
CONSENSUS

In Bitcoin, the mean time for a node to receive a complete block is 12.6 seconds. After 40 seconds, 95% of the nodes have seen that block. See “Information propagation in the bitcoin network” by Decker and Wattenhofer.

There is a roughly a 10 minute delay between block creations

So, only on rare occasions would a node receive two blocks at about the same time. If a node does see two blocks, the next block will likely resolve the issue.

A Fork – Suppose you are inside a node

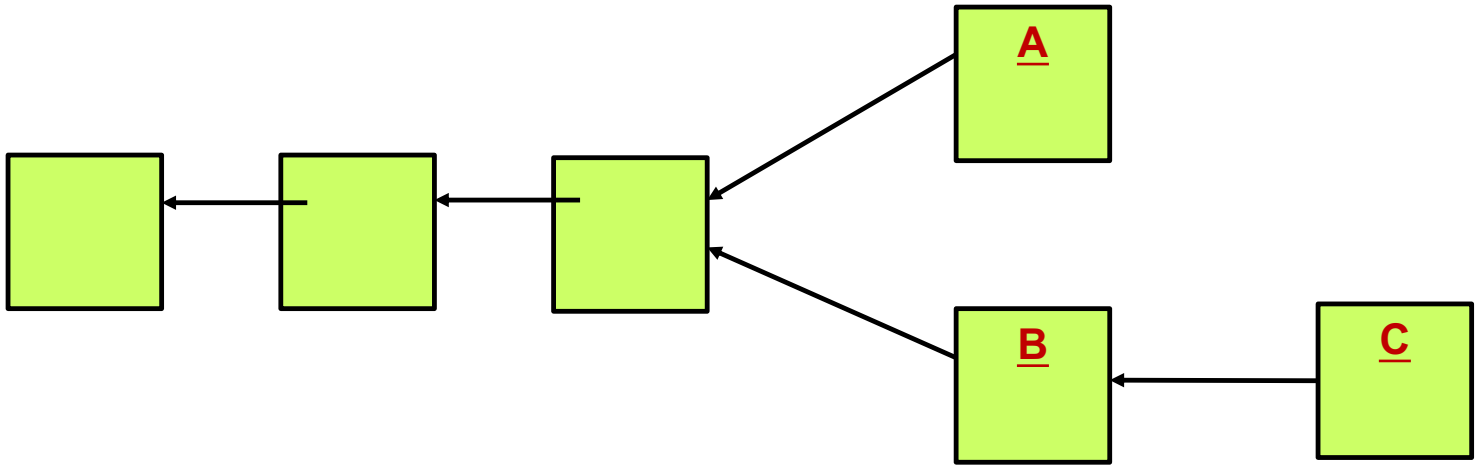


This node will attempt to build on block A because block A arrived first.

Suppose a new block C arrives.

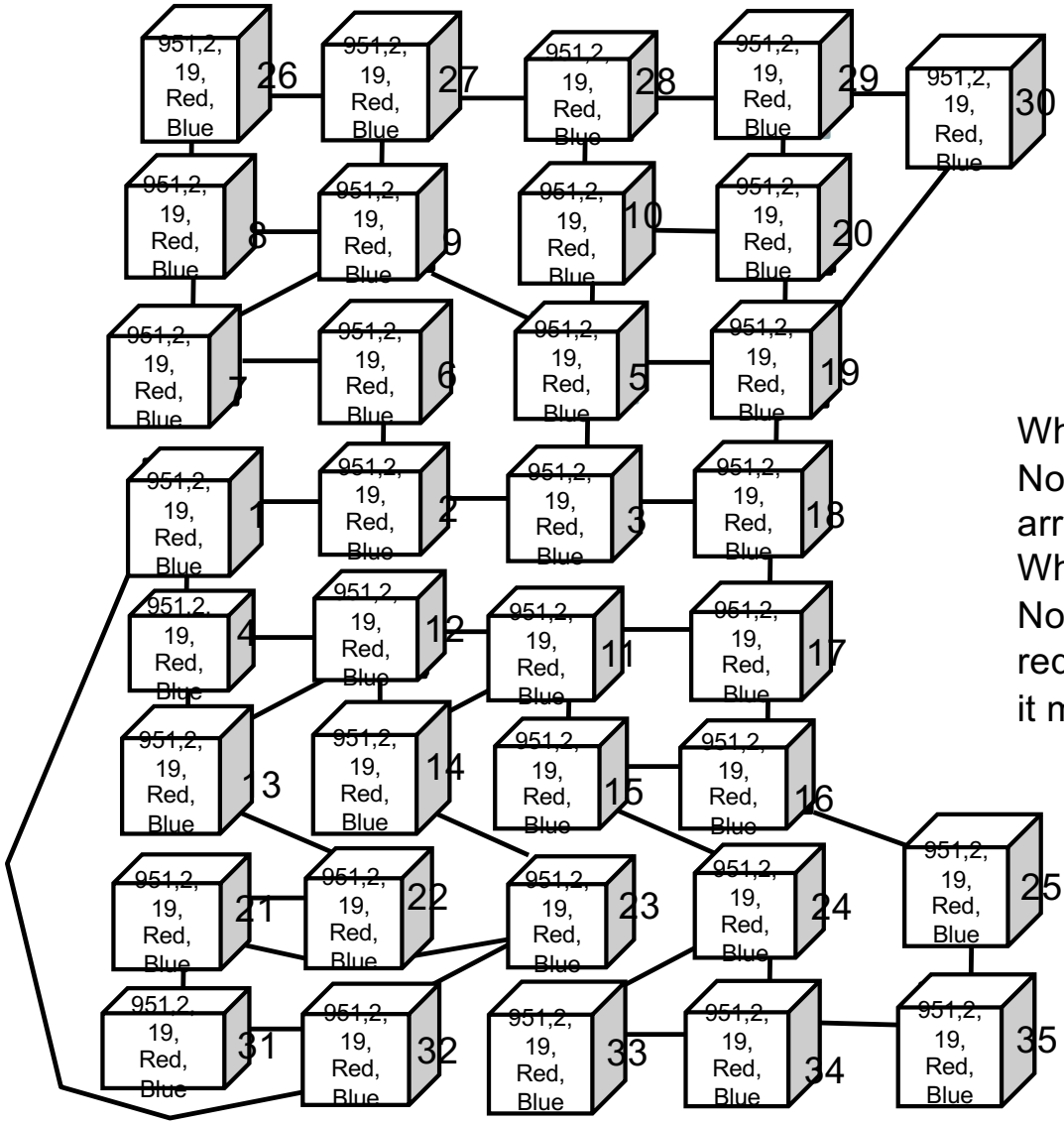
This node will now begin to build on block C. It is the longer chain.

A 51% Attack

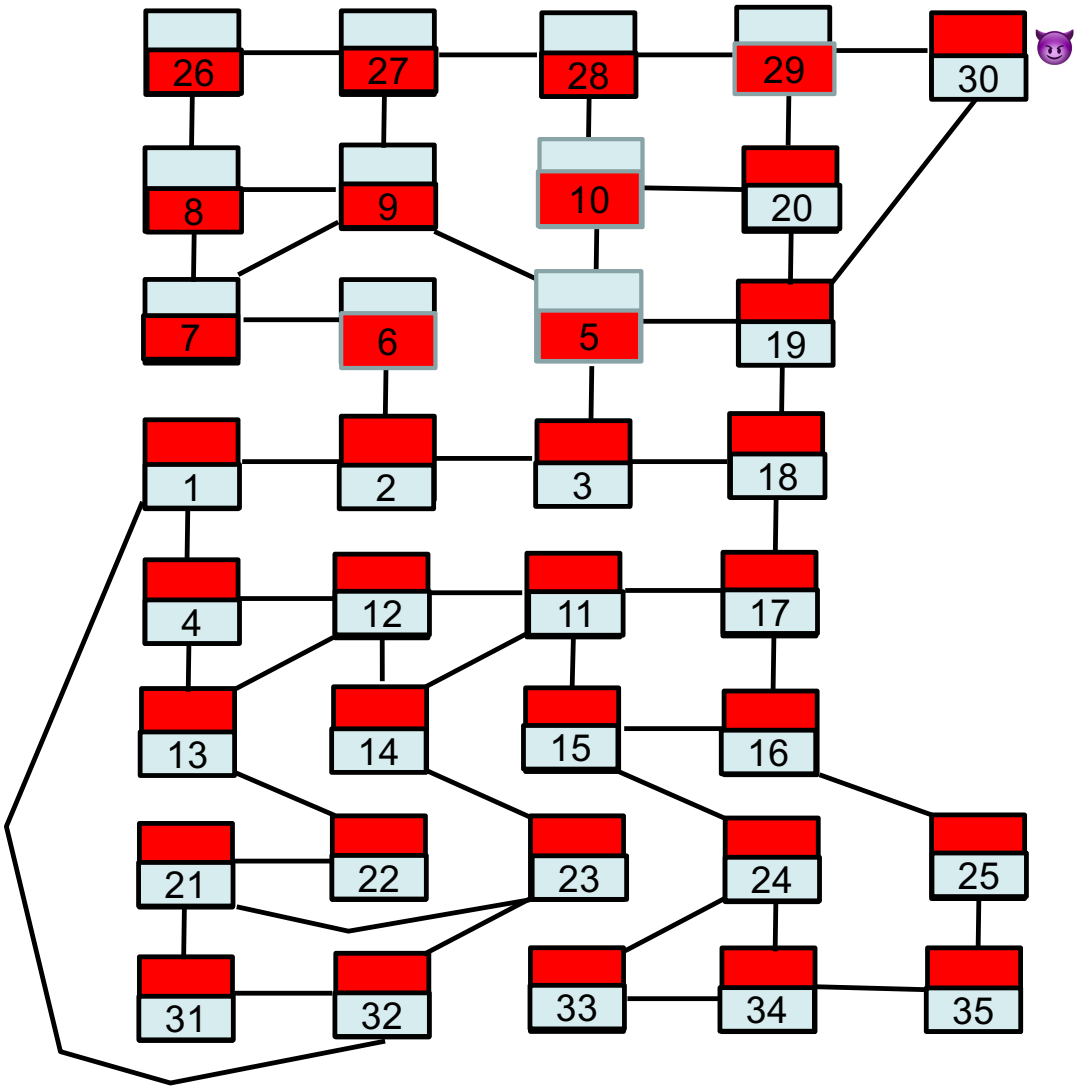


If block B was computed in private, it could be sent out after the transactions in A were confirmed and acted upon.
If a bad player could out run the entire network (with 51% of the hash power) then they could deliver node C and node D and node E, and so on, and effectively erase the transactions in A. Goods may have been delivered without payment.

Bitcoin has never suffered such an attack. Alt coins have.

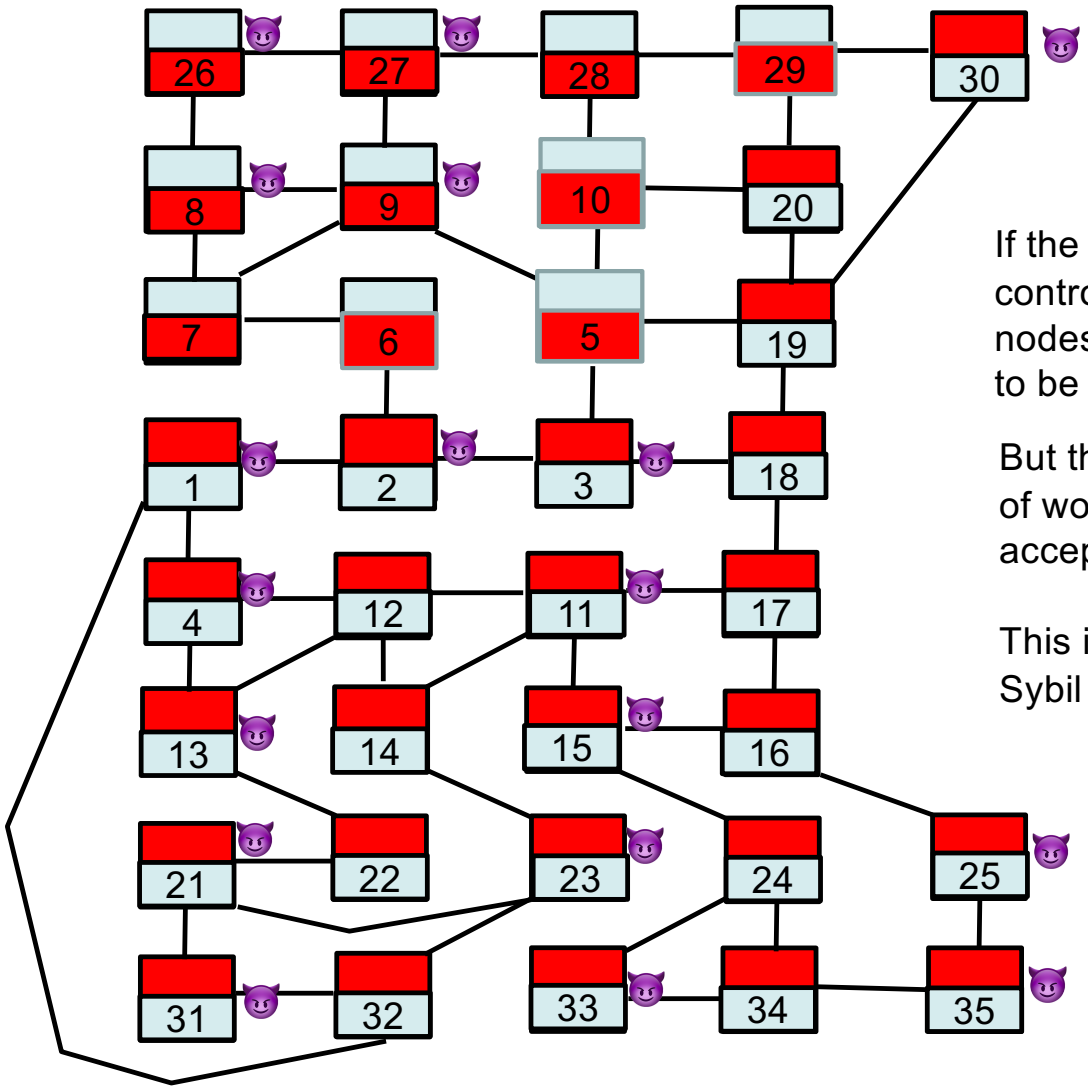


What if we lose a transaction on the network?
No problem, it will likely be found in the next block to arrive.
What if we lose a block on the network?
No problem, if a node does not receive a block, it will request it when it receives the next block and realizes it missed one.



Suppose peer 30 wants its order
without computing proof of work.
3451,
2,
30,
Red message,
Blue message

19 and 29 will check the proof of work and drop 30's request.



If the bad guys collude and control more than half the nodes then they are likely to be able to pick the next block.

But they need to compute proof of work for the honest nodes to accept their decision.

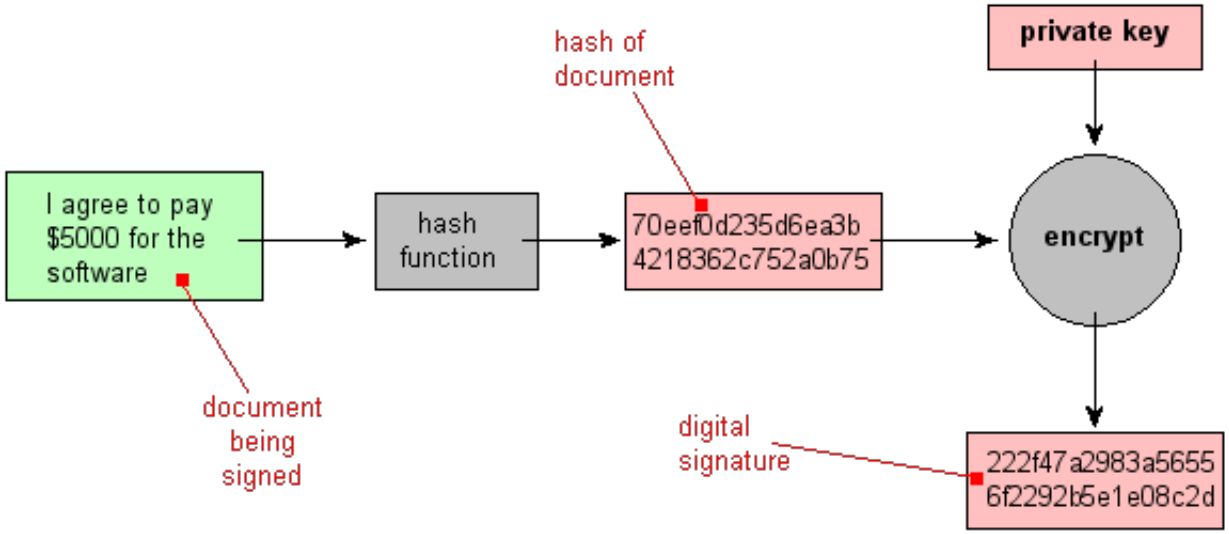
This is possible but this makes a Sybil attack very very expensive.

In general, we want to give the good guys easy problems to solve (checking proof of work) and the bad guys hard problems to solve (computing proof of work).

5. Nodes accept the block only if all transactions in it are valid and not already spent.

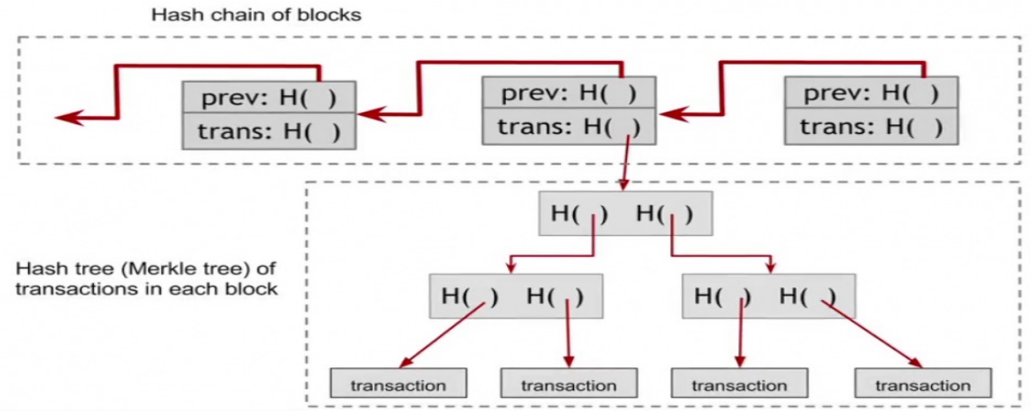
6. Nodes express their acceptance of the block by working on creating the next block in the chain, using the hash of the accepted block as the previous hash.

We need more cryptography (Signatures)



It's easy to 1) sign the document and 2) verify the signature with the public key of the signer. It would (presumably) be very very hard to compute the signer's private key from the signer's public key.

We need more cryptography (Blockchain and Merkle Trees)



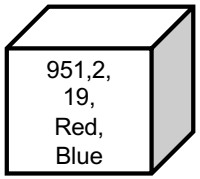
From Wikipedia

Easy to hash the tree and add transaction. Very very hard to modify a transaction once it is deep in the chain. Proof of work needs to be recomputed for each following block and it must be done sequentially.

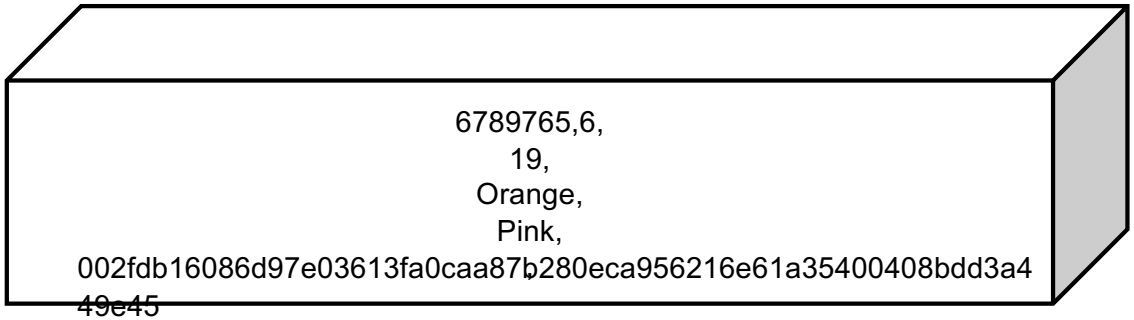
Quiz: Which of the blocks on the right can be the next block in the chain of length 2?
Format: Nonce, Difficulty, id, Tx1, Tx2, HashPointer

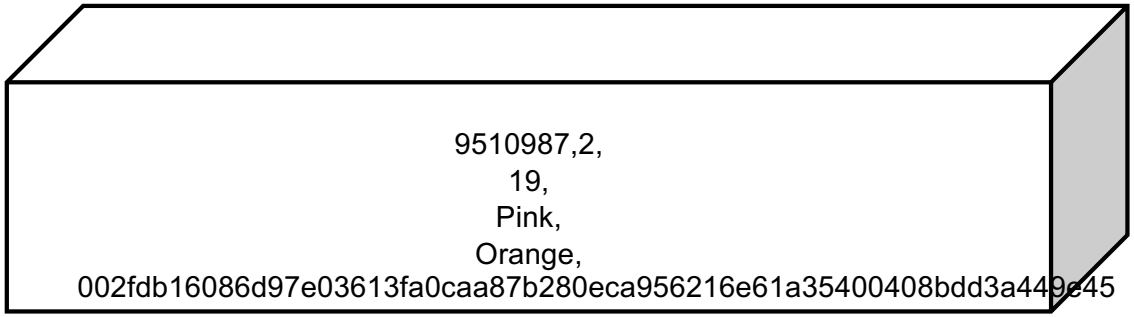
Use:
https://www.xorbin.com/tools/sha256-hash-calculator

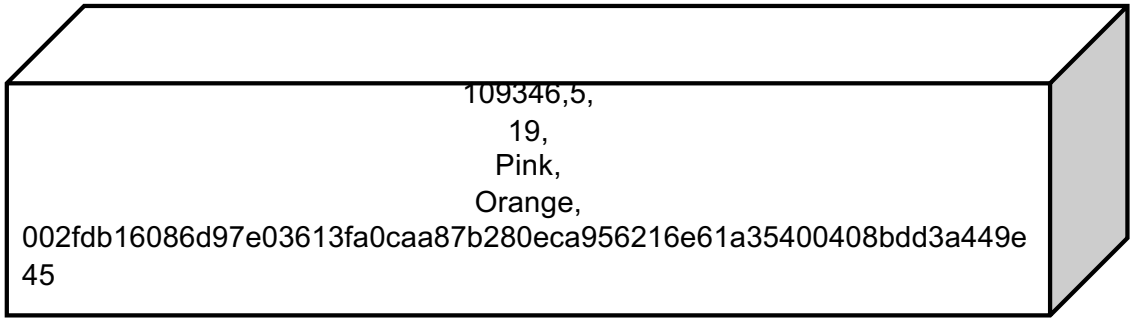
Genesis Block



002fdb16086d97e03613fa0caa87b280eca956216e61a35400408bdd3a449e45







Bibliography

In Search of an Understandable Consensus Algorithm
Diego Ongaro and John Ousterhout Stanford University

Bitcoin: A Peer-to-Peer Electronic Cash System
Satoshi Nakamoto